

팍리스 실무형 일경험 프로그램

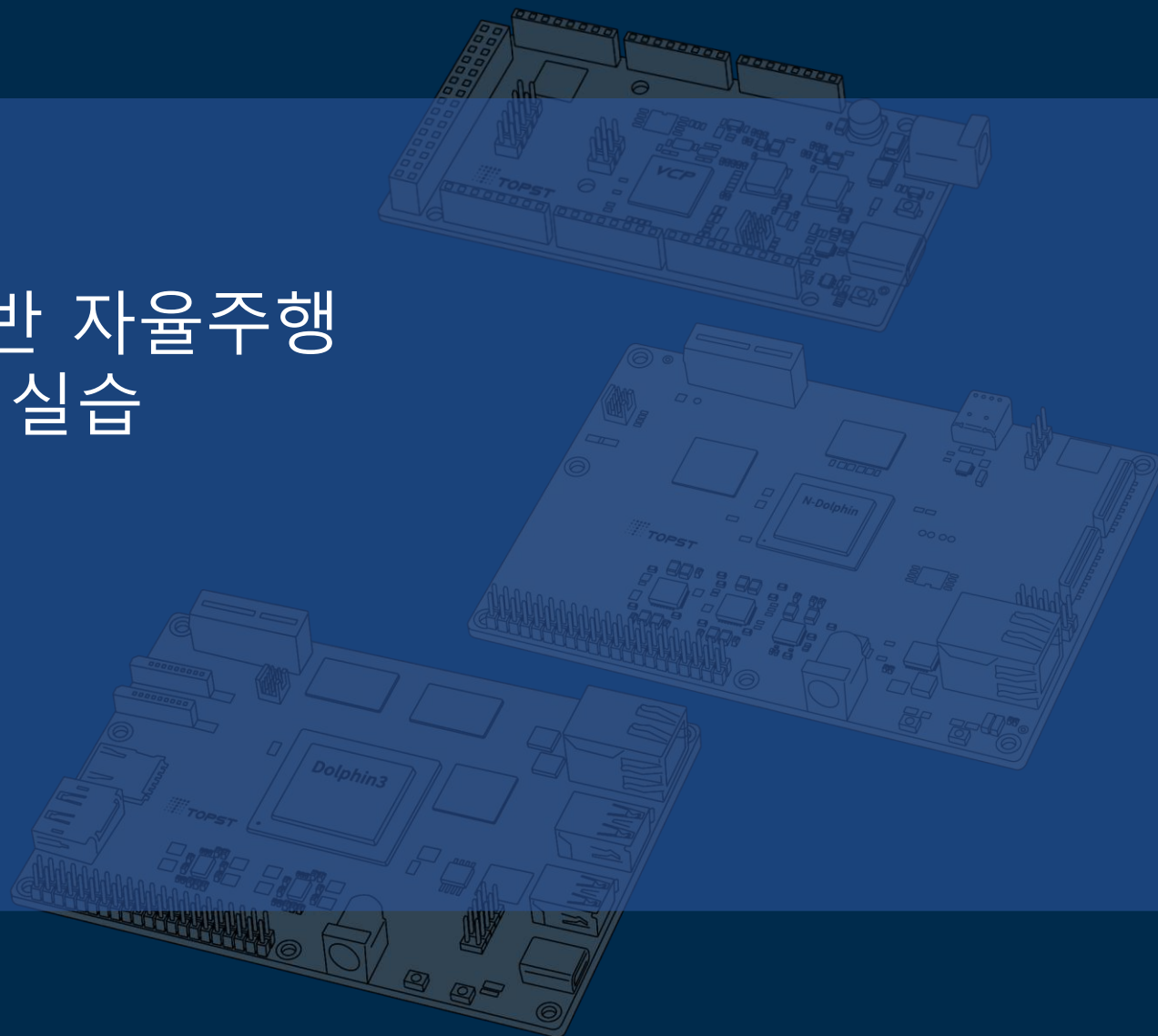
SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



6일차

02 AI Modeling 기초

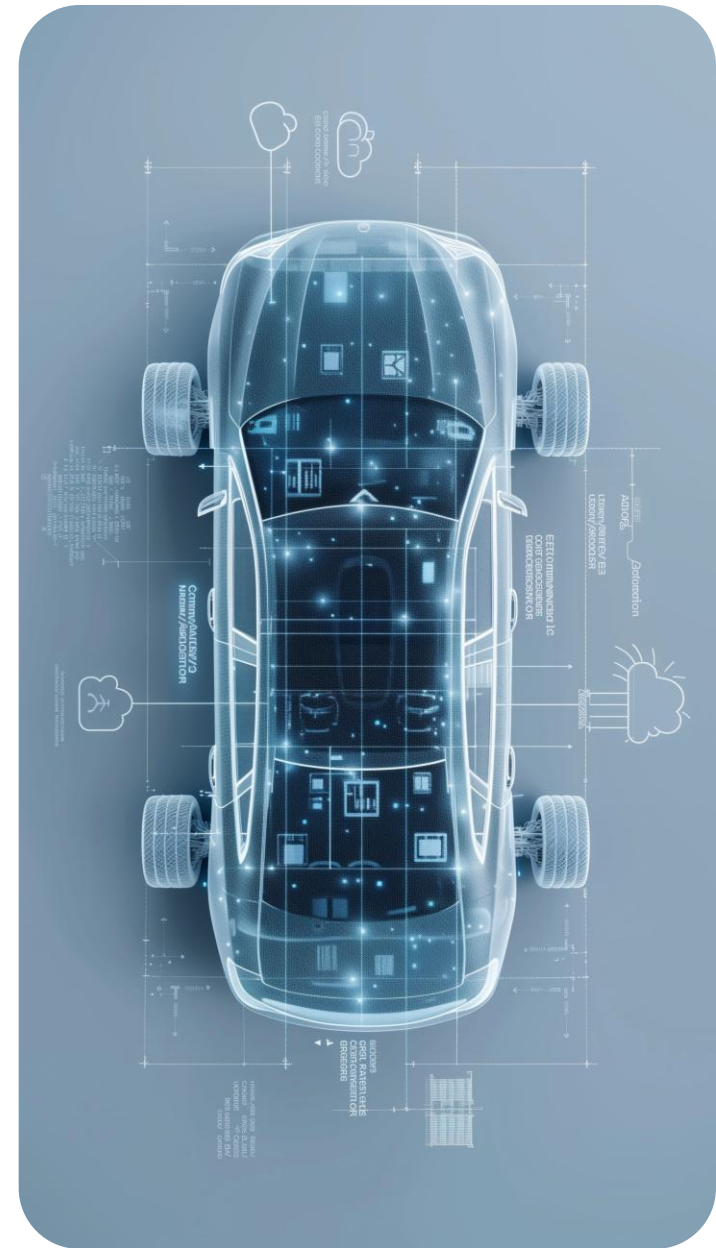
Telechips TOPST



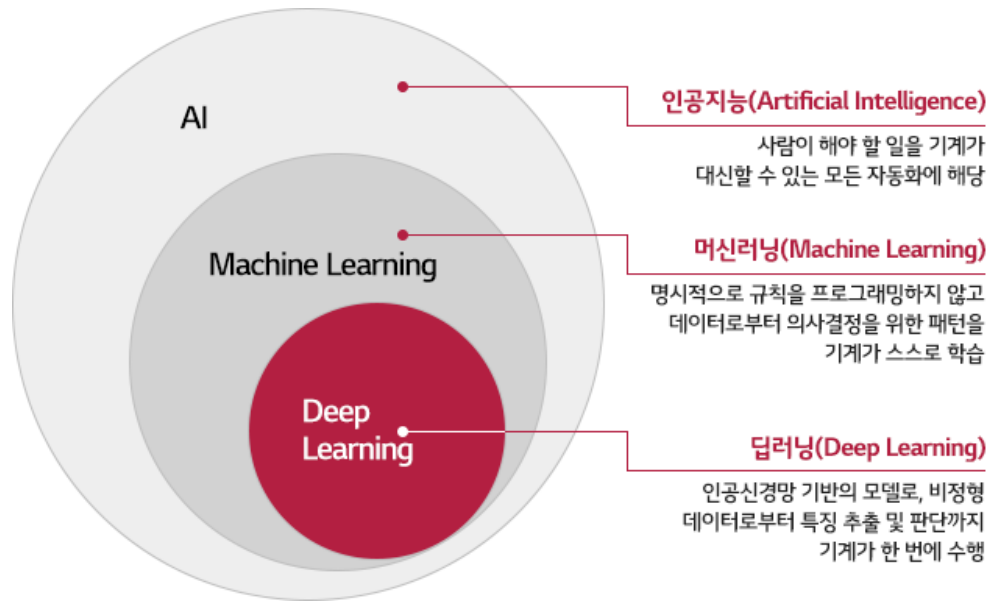
목차

Table of Contents

AI Modeling 기본	01
tc-nn-toolkit 이해	02
tc-nn-toolkit 환경 세팅	03
Lenet5 모델 학습	04
Lenet5 모델 변환	05
모델 추론 - Lenet5	06



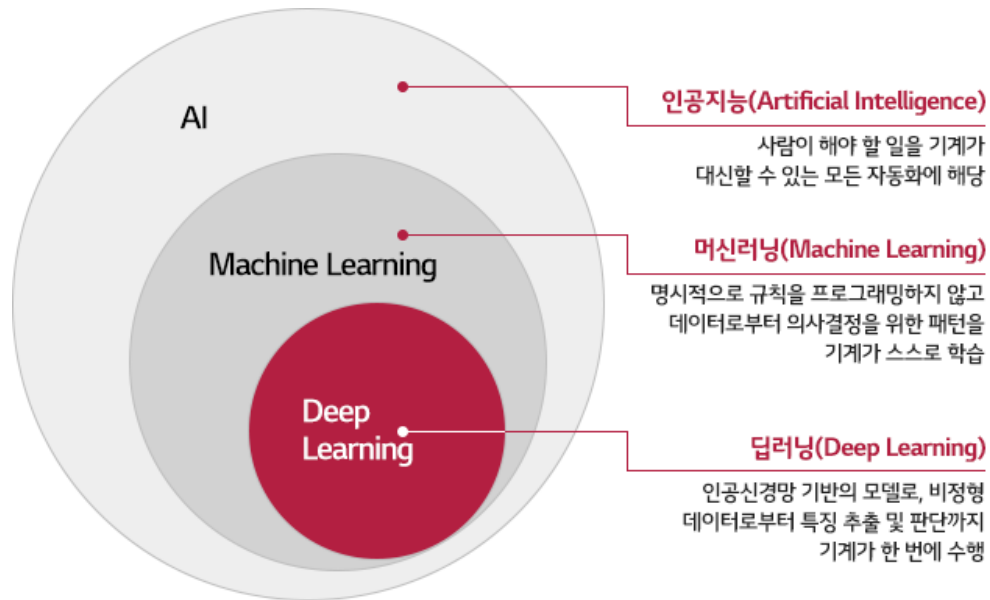
AI Modeling 기본



1. 인공지능, 머신러닝, 딥러닝의 관계

- 인공지능 : 인간의 사고나 판단, 인식 능력을 기계가 수행하도록 만드는 개념
- 머신러닝 : 인간의 학습 능력과 같은 기능을 컴퓨터에 부여하기 위한 기술
- 딥러닝 : 머신러닝의 한 종류로 신경망을 여러 층으로 쌓아 복잡한 패턴을 학습하여 판단하는 기술

AI Modeling 기본

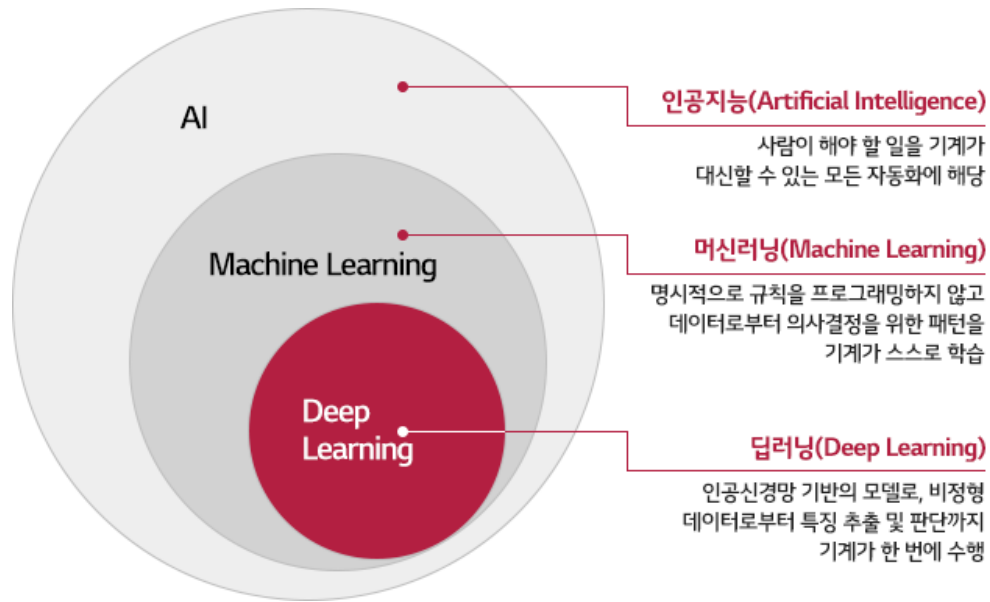


1. 인공지능, 머신러닝, 딥러닝의 관계

- 주요 활용 분야

- 스마트폰 음성 비서 (시리, 빅스비)
- 자율주행 자동차, 드론
- GPT, 제미나이 등 (LLM)
- 생성형 AI

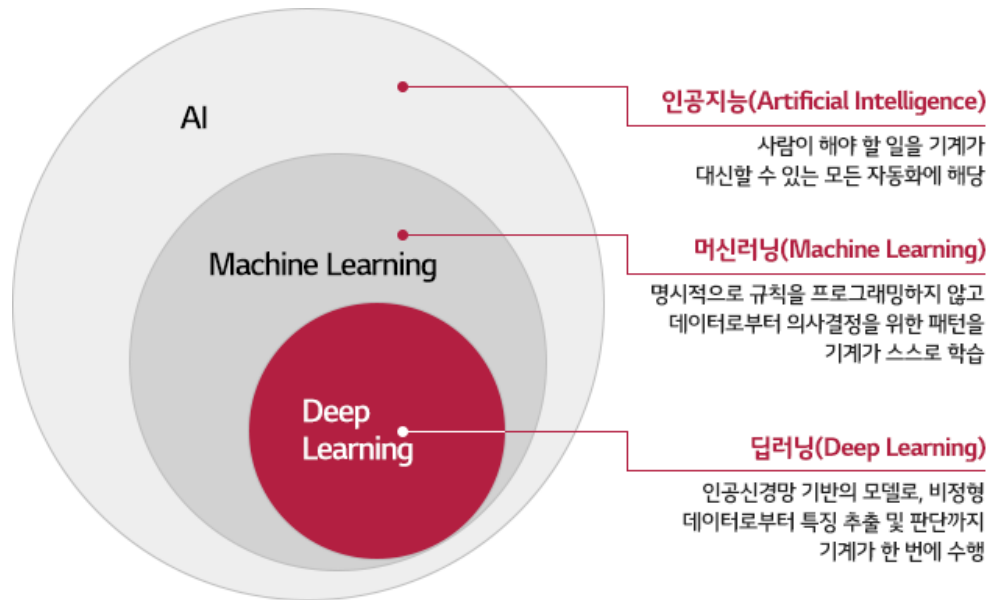
AI Modeling 기본



1. 인공지능, 머신러닝, 딥러닝의 관계

- 머신러닝이 등장하게 된 배경
- 기존의 AI는 방법이 아니라 목표이며 학습할 필요가 없이 탐색, 계획, 논리 추론 등으로 사람이 해야 할 지능적 판단만을 수행하도록 하였음
- 사람이 규칙을 다 적을 수 없고, 데이터는 많은데 규칙을 모르는 경우가 생겨 머신러닝이 등장
- 즉, 명시적인 규칙 대신 데이터로부터 규칙을 학습하는 방법

AI Modeling 기본

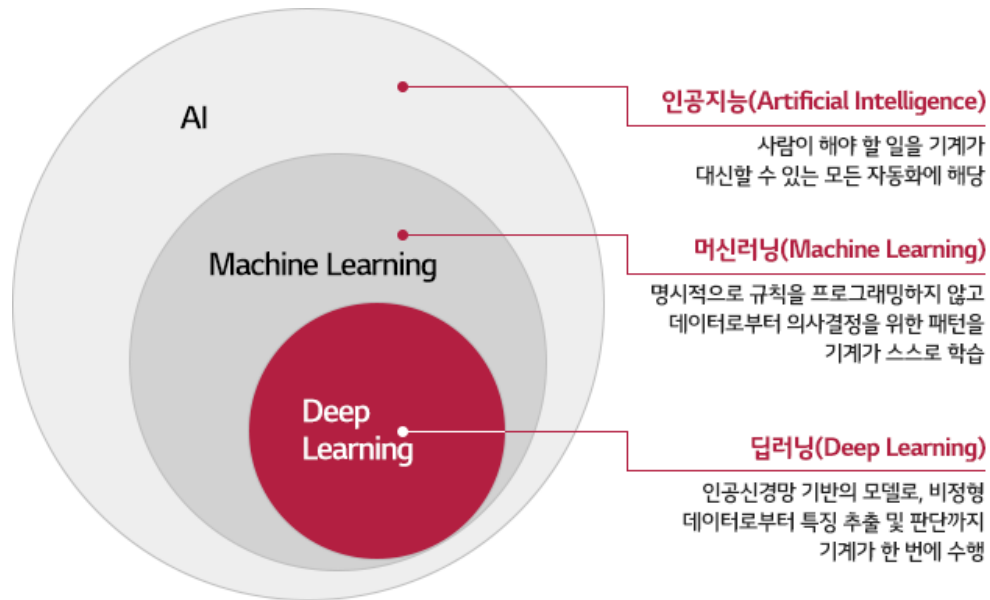


1. 인공지능, 머신러닝, 딥러닝의 관계

- 머신러닝의 한계

- 특징을 사람이 설계한다는 한계점
- 특정 설계가 사람의 경험과 도메인 지식에 의존
- 이미지/음성/자연어 처럼 복잡한 데이터에서는 좋은 특징을 만들기 어려움

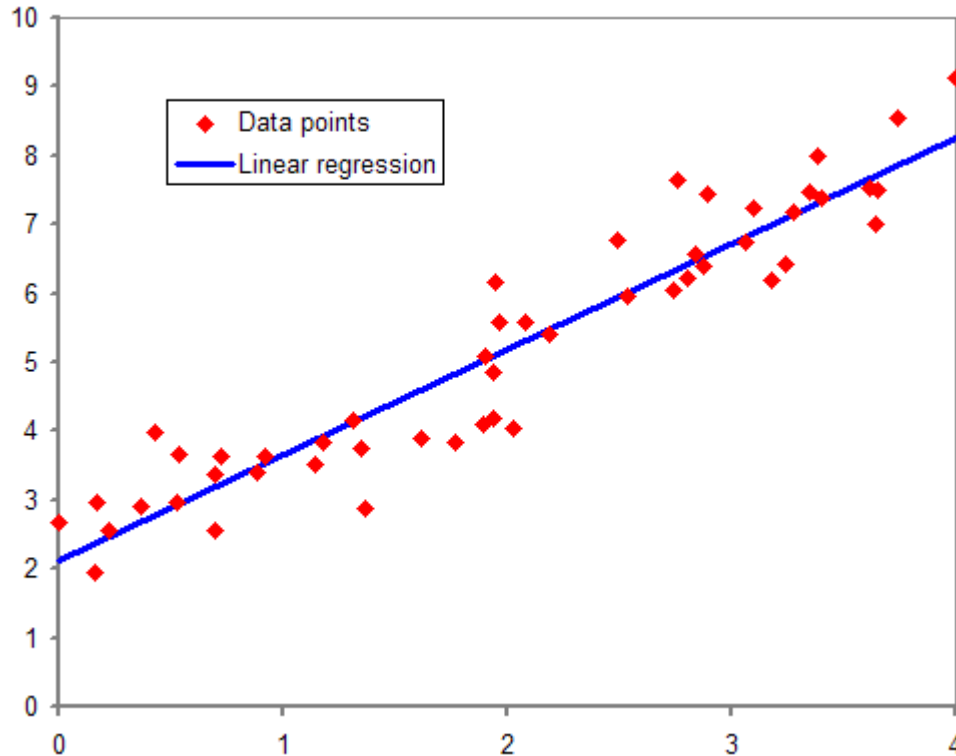
AI Modeling 기본



1. 인공지능, 머신러닝, 딥러닝의 관계

- 딥러닝 : 특징도 모델이 학습
- 딥러닝은 신경망이라고 하는 것을 여러 층으로 쌓아, 특징 추출과 판단을 동시에 학습하는 머신러닝 기법

AI Modeling 기본



2. 인공지능 모델이란

- 입력(Input) -> 함수(Function) -> 출력(Output)

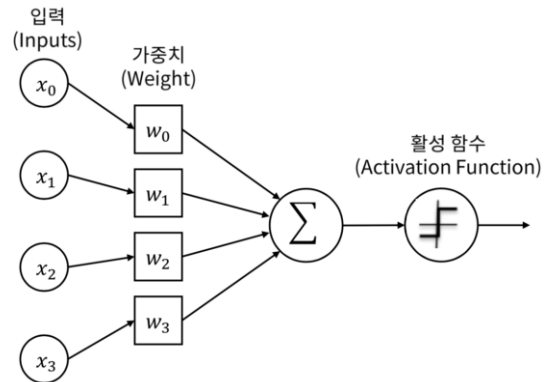
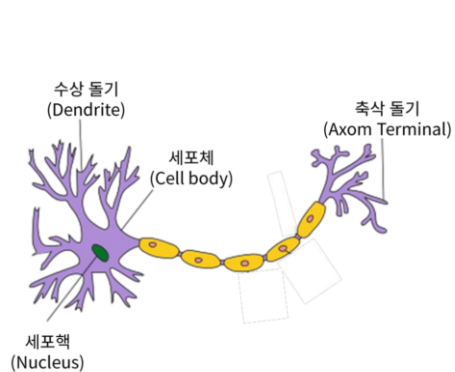
$$\text{ex) } y = w * x + b$$

- 모델이란 입력과 출력 사이의 관계를 표현하는 함수

- 입력 x 를 받아 출력 y 를 만드는 함수

- AI 모델 개발은 "파라미터(w, b)"를 찾는 과정

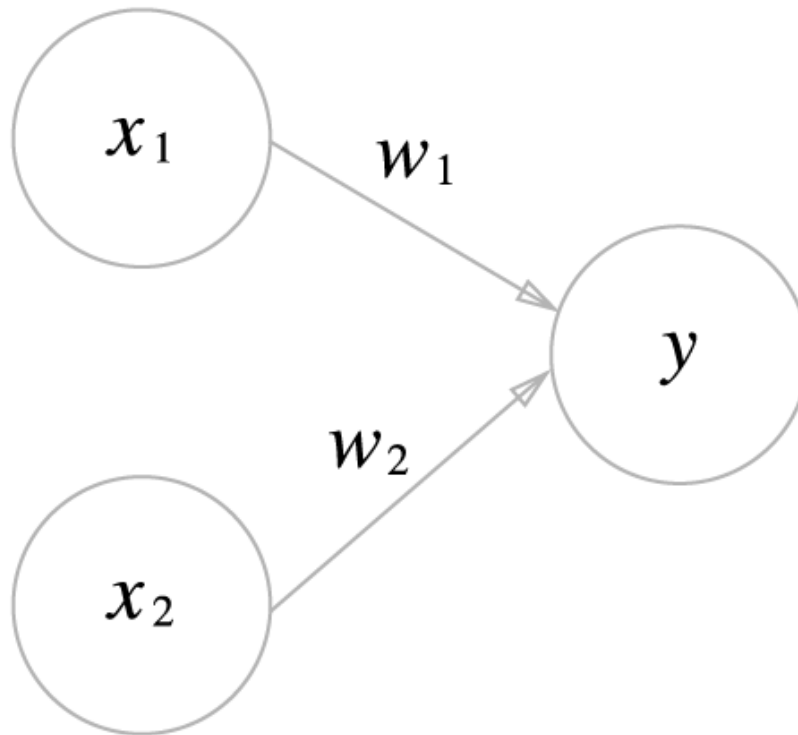
AI Modeling 기본



3. 퍼셉트론 이란?

- 인간의 뉴런을 모방하여 만든 최초의 인공 신경망 모델
- 여러 입력을 받아 하나의 출력을 내는 단순한 형태
- 기존의 선형 회귀에서 활성화 함수를 추가해 출력 형태를 바꿈
- 가중치를 학습하여 데이터를 선형적으로 분류하는 알고리즘

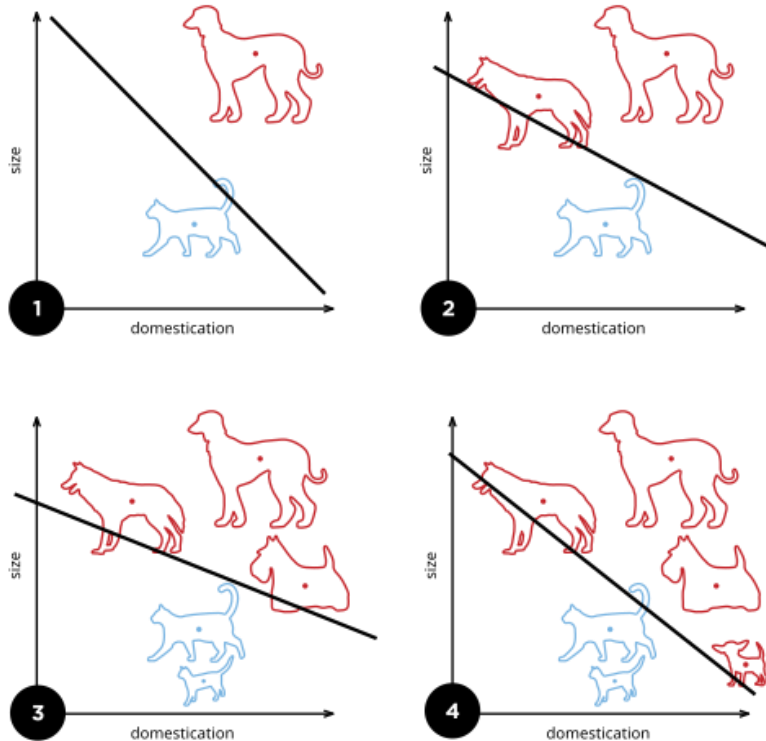
AI Modeling 기본



3. 퍼셉트론 이란?

- 퍼셉트론에서는 가중치(w)라고 부르는 weight가 입력신호에 부여되어 연산을 진행하고 임계값을 넘으면 1을 출력하고 아닐 경우 0 또는 -1 출력
- 각 입력신호에는 고유한 weight가 부여되고 weight는 각 입력이 결과에 미치는 영향력을 나타냄

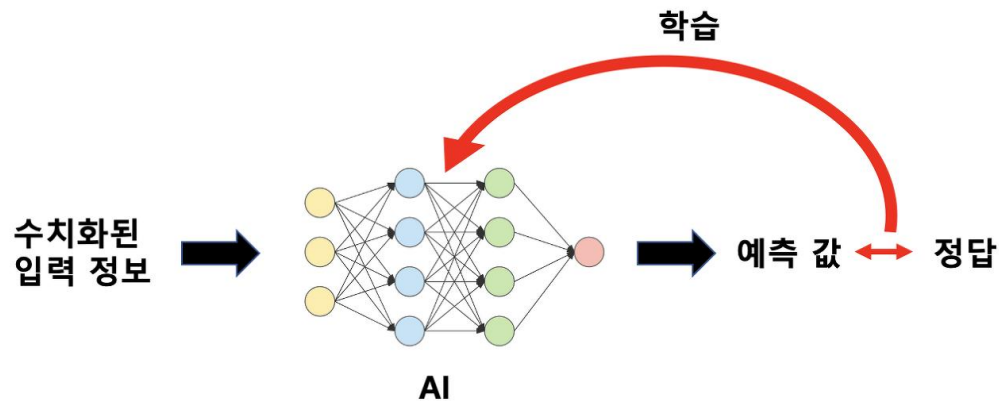
AI Modeling 기본



3. 퍼셉트론 이란?

- 선형 분류는 그림과 같이 선으로 분류하는 것을 의미
- 학습이 반복될 수 록 기울기가 달라지며 가중치 값을 계속 조정
- 퍼셉트론은 데이터를 하나의 직선으로만 나눌 수 있음
- 복잡한 패턴을 표현하기에는 한계가 있음

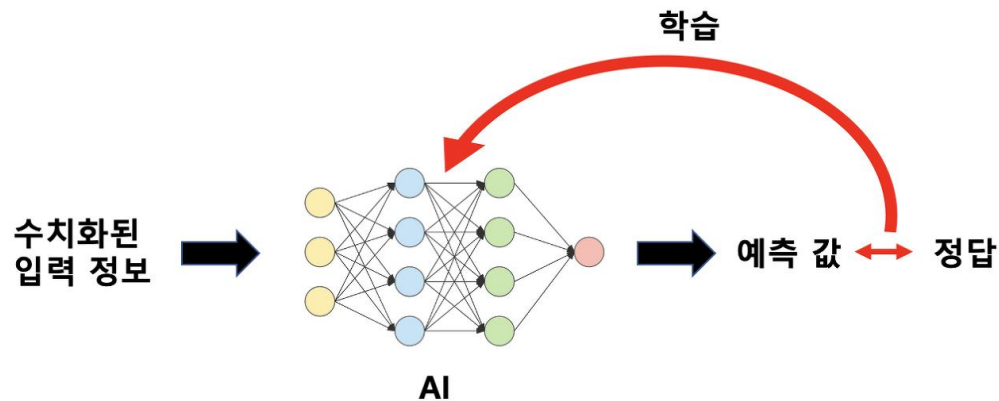
AI Modeling 기본



4. 학습 이란?

- 데이터를 잘 설명하도록 모델을 조정하는 과정
- 즉, 예측, 오차계산, 파라미터 수정 과정을 계속해서 반복하는 과정
- 파라미터 = 모델이 학습을 통해 바꾸는 값
- 초기에는 w 와 b 의 값을 무작위로 설정
- 초기 모델 : $y = w * x + b$

AI Modeling 기본



4. 학습 이란?

예측과 실제의 차이

- 실제 데이터와 예측 값 사이의 오차 계산
- Ex) 실제 몸무게 60
모델 예측 55
오차 -> $60 - 55 = 5$
- 이 오차를 수학적으로 표현 -> 손실 함수
- 데이터가 1개가 아니라 수천 개 이기 때문에
전체 오차를 하나의 숫자로 수치화 필요
- 오차의 부호가 상쇄되지 않도록 제곱, 절댓 값
사용

AI Modeling 기본

$$MSE = \frac{1}{N} \sum_i^N (pred_i - target_i)^2$$
$$RMSE = \sqrt{\frac{1}{N} \sum_i^N (pred_i - target_i)^2}$$
$$MAE = \frac{1}{N} \sum_i^N |pred_i - target_i|$$

4. 학습 이란?

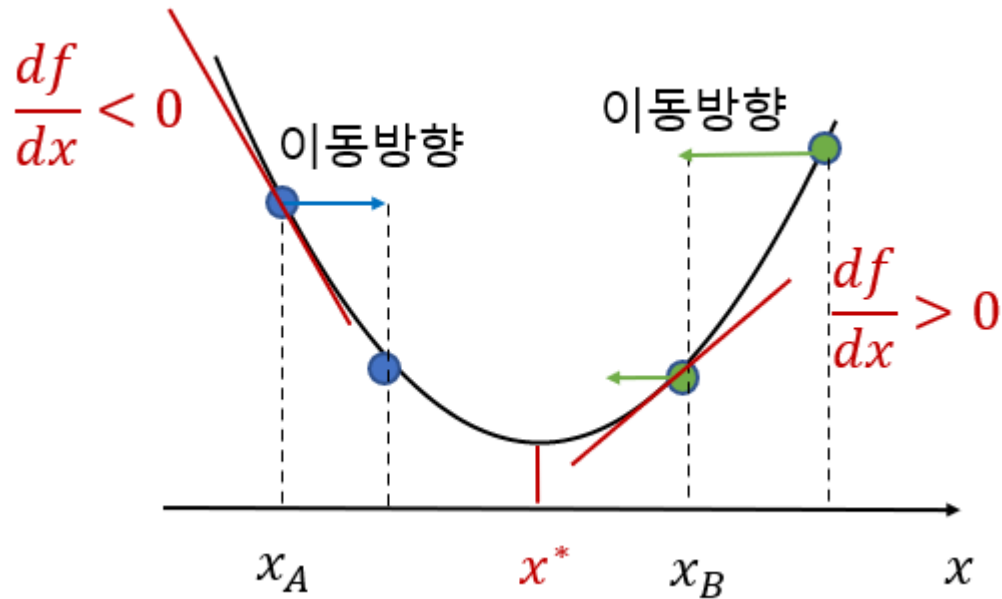
손실함수의 정의

- 손실 함수는 모델이 얼마나 틀렸는 지에 대한

$$Loss = (y_{true} - y_{pred})^2$$

- 다음과 같이 오차를 구하고 평균 값을 구해 전체 Loss를 구함
- 오차가 클수록 Loss가 커지고 완벽히 예측하면 Loss는 0이 된다.
- 모델 학습은 Loss를 최소화하기 위한 w, b의 값을 찾는 것

AI Modeling 기본

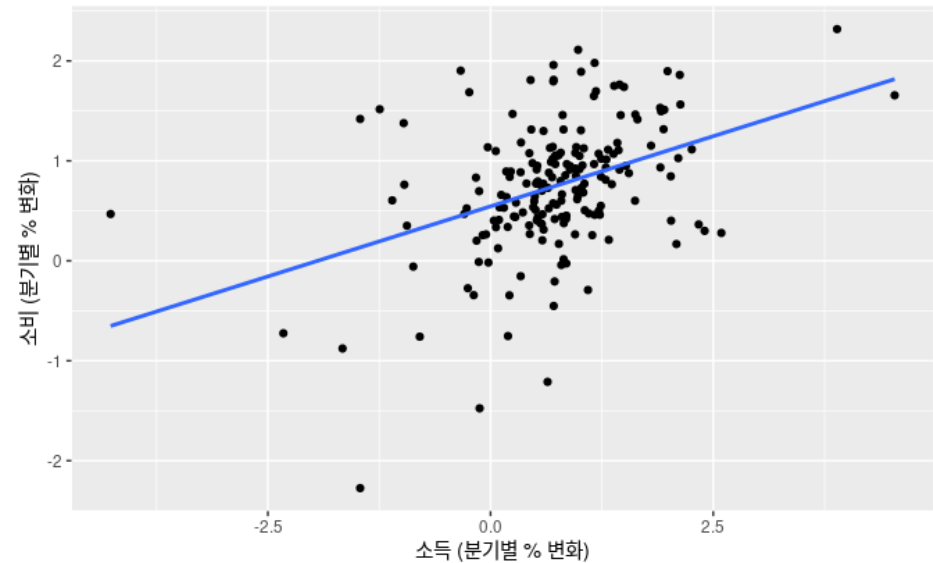
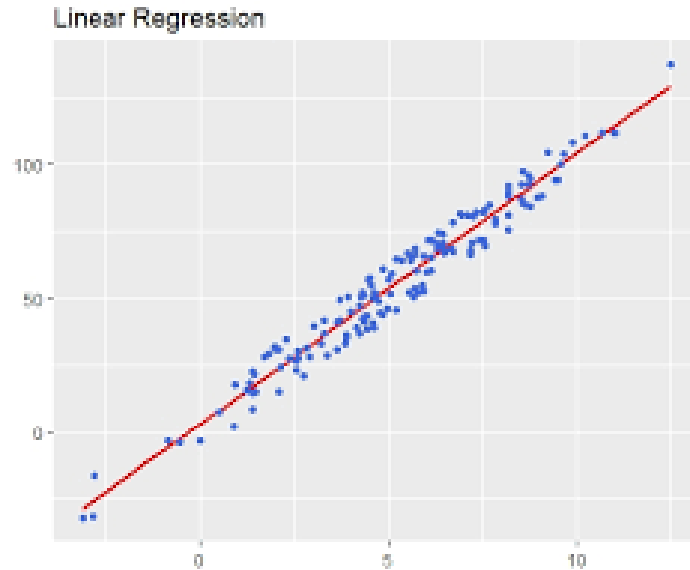


4. 학습 이란?

경사 하강법

- 손실함수를 최소화하기 위한 반복적 최적화 알고리즘
- 모델이 틀리게 예측할 때 마다 오차를 계산
- 손실 함수의 기울기를 이용해 Loss를 줄이는 방향으로 w, b 파라미터를 조정
- 여기서 x 축은 파라미터(w, b)를 의미
- 이 과정을 반복해 점점 더 정확한 예측을 하게 됨

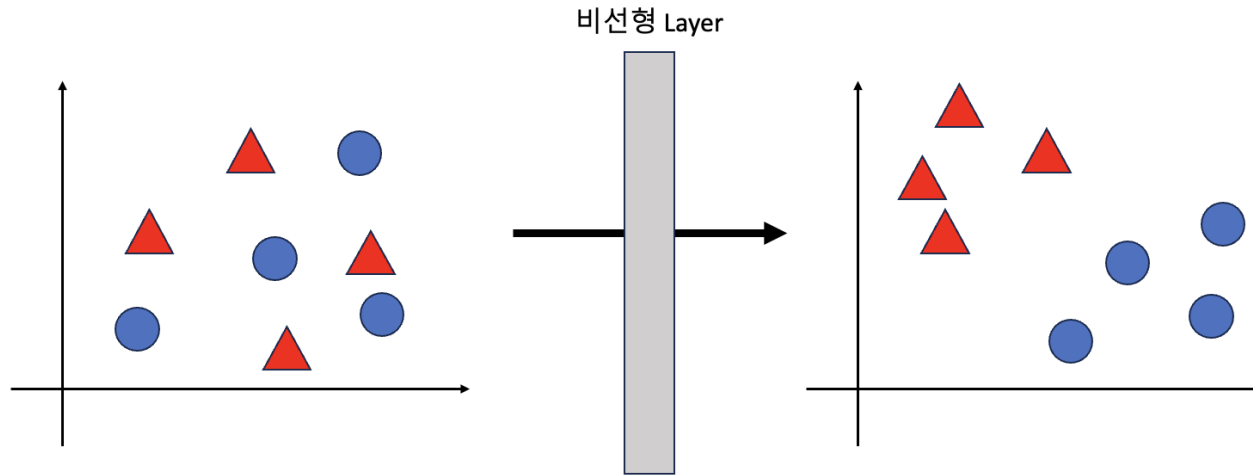
AI Modeling 기본



5. 선형 / 비선형 모델이란?

- 선형 모델이란 "머신러닝 공식에서 계수들이 선형결합의 관계에 있을 때의 모델"을 의미
- 입력 x 와 출력 y 가 직선 관계(비례 관계)로 표현되는 모델
- ex) 선형 회귀, 퍼셉트론

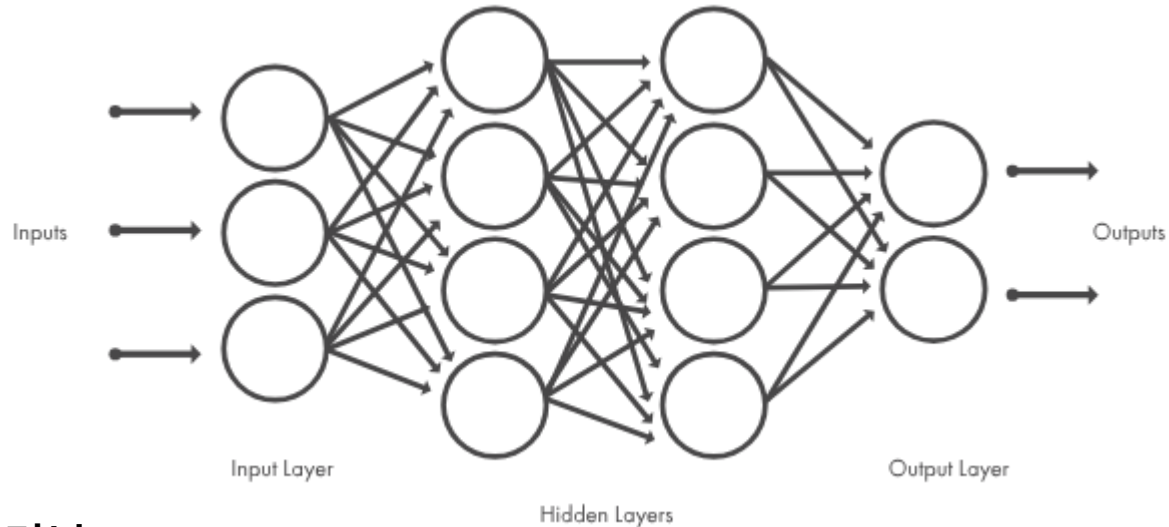
AI Modeling 기본



5. 선형 / 비선형 모델이란?

- 비선형 모델이란 입력과 출력 관계가 곡선 형태이거나 단순한 선형 결합으로 설명되지 않는 모델
- Ex) 로지스틱 회귀, 신경망 등
- 비 선형 모델이 필요한 이유
- 실제 데이터는 직선으로 설명되지 않는 경우가 많음
- 복잡한 패턴, 경계, 곡선 형태를 학습할 수 있음

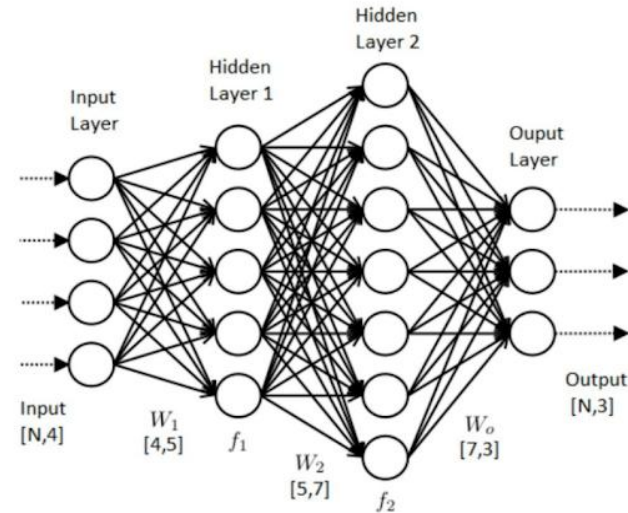
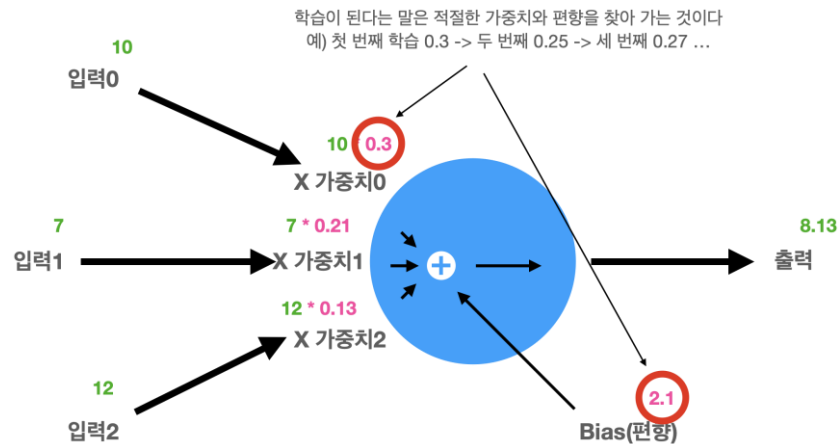
AI Modeling 기본



6. 딥러닝

- 데이터로부터 스스로 특징을 학습하는 다층 인공 신경망
- 기존 머신 러닝은 사람이 특징을 설계해야 했지만, 딥러닝은 데이터를 통해 자동으로 특징을 추출
- 층이 깊어질수록 더 복잡한 특징(모양, 패턴, 의미)을 단계적으로 학습

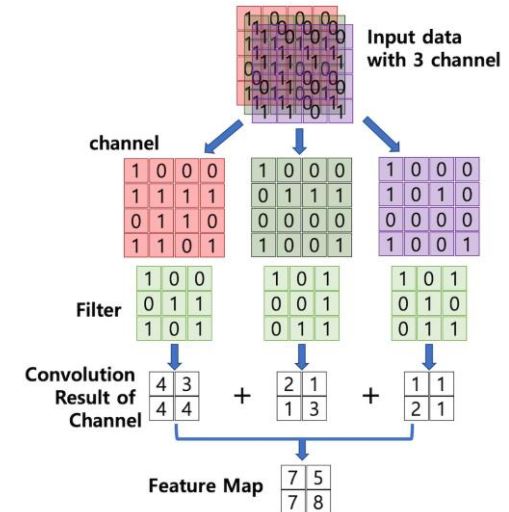
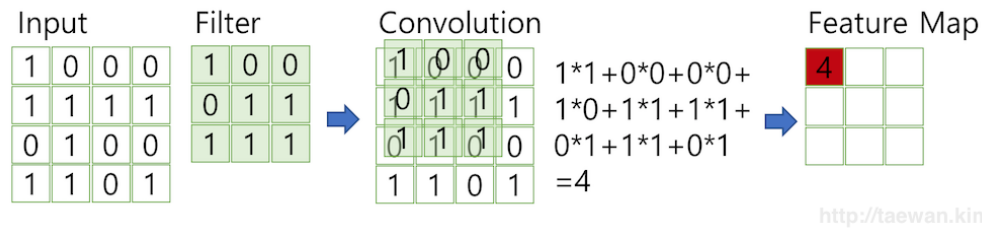
AI Modeling 기본



6. 딥러닝

- 핵심 특징 : 다층 구조를 통해 비선형 관계를 학습하고, 자동으로 특징 추출
- 학습 방식 : Loss 최소화 -> 경사 하강법으로 가중치(w) 갱신
- 장점 : 높은 정확도, 다양한 데이터 타입에 적용 가능
- 단점 : 데이터, 연산량 많음, 내부 구조 해석 어려움
- 대표 모델 : CNN, RNN, Transformer, AutoEncoder

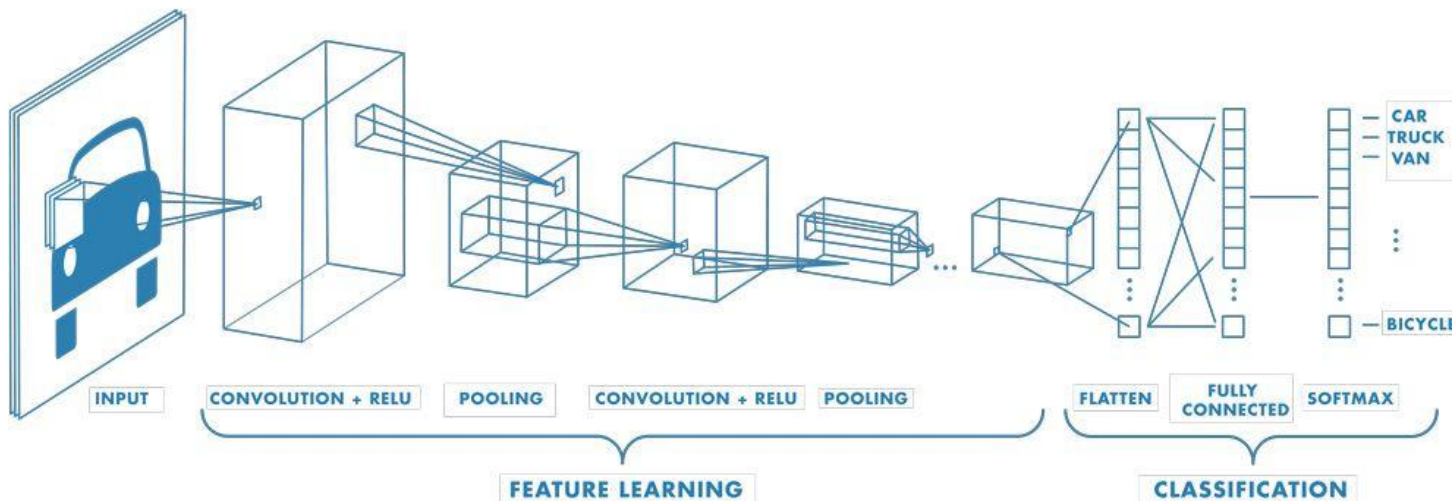
AI Modeling 기본



7. CNN

- 이미지에서 패턴을 찾아내도록 설계된 신경망
- 핵심 : 합성곱(Conv) + 비선형 활성화 함수(ReLU, SiLU 등) + 풀링 / 스트라이드
 - Convolution(합성곱) : 작은 필터가 이미지 전체를 훑으며, 모서리, 점, 무늬 같은 패턴을 찾아내는 계산
 - 비선형 활성화 함수 : 추출된 특징에 비선형성을 주어 더 복잡한 패턴을 표현
 - 풀링 / 스트라이드 : 특징 맵을 요약(다운 샘플링) 하여 위치 변화에 둔감하게 만들고 연산량을 줄임

AI Modeling 기본

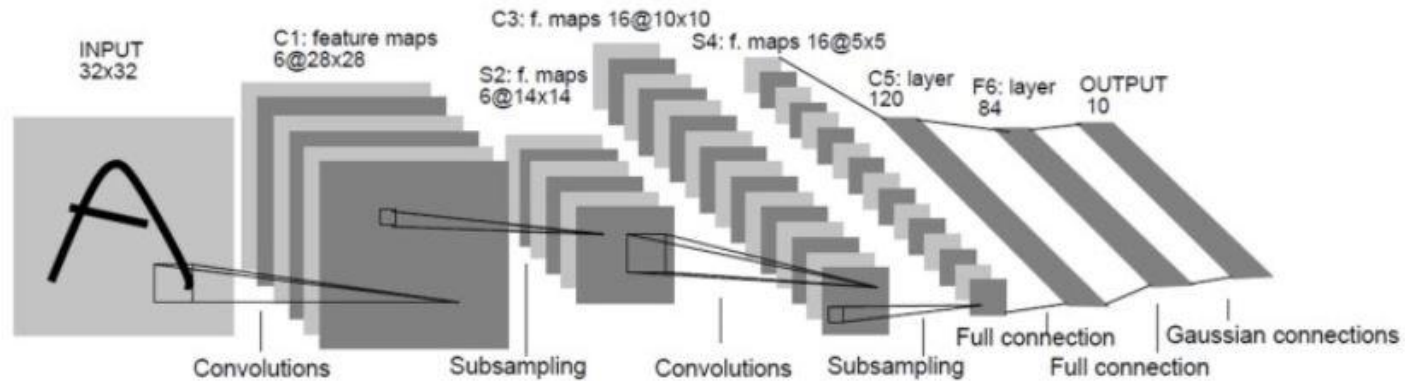


7. CNN

- 왜 이미지에 강한가?

- 주변 픽셀끼리 서로 연관성이 있어 경계, 무늬, 텍스처가 필터 안에서 결정되는 경우가 많다.
- 반복적으로 나타나는 패턴은 위치만 다를 뿐 모양은 유사함. 한번 배운건 어디서든 인식 가능
- FC는 입력의 모든 픽셀 x 출력을 연결해서 가중치가 폭증해 연산량 많음 CNN은 파라미터가 훨씬 적어 학습 안정 및 메모리 절약

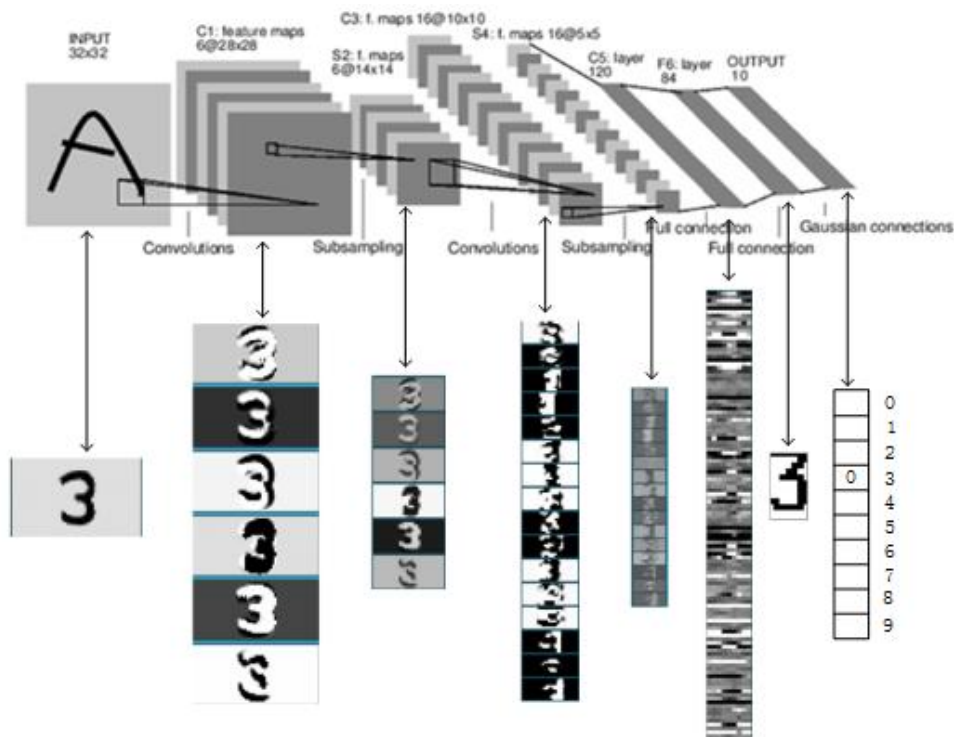
AI Modeling 기본



8. LeNet-5란?

- 초기 CNN 모델로 손 글씨 숫자(MNIST) 인식에 사용됨
- 오늘날 CNN의 기본 골격 (Conv->Pool->Conv->FC)을 자리 잡게 함
- 우편번호, 숫자 인식 등에 사용됨
- 0~9까지 총 10개의 숫자 인식 가능

AI Modeling 기본

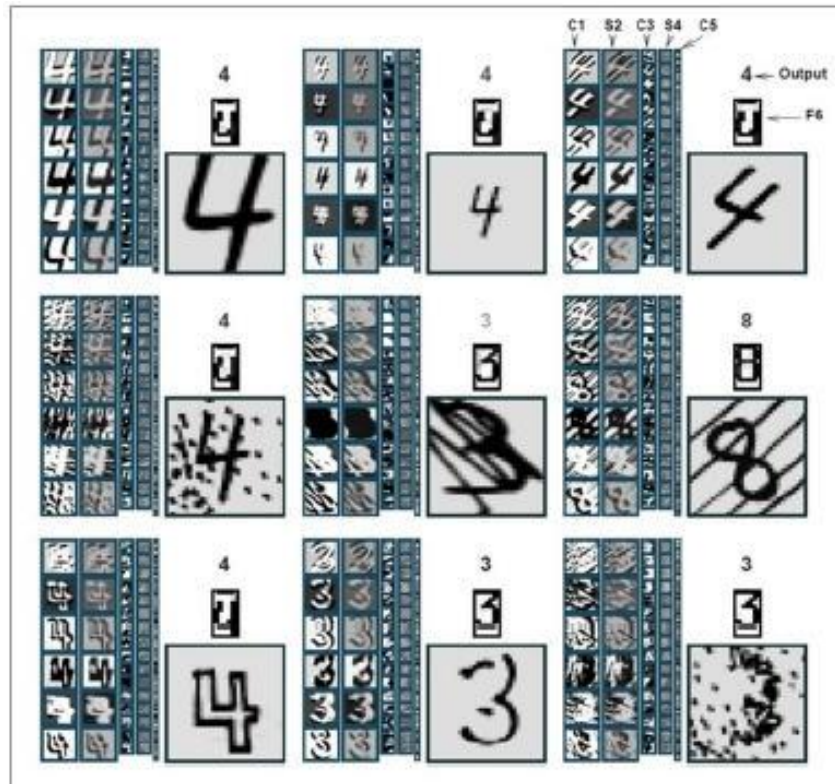


8. LeNet-5의 구조

- Input $32 \times 32 \times 1 \rightarrow C1 \rightarrow S2 \rightarrow C3 \rightarrow S4 \rightarrow C5 \rightarrow F6 \rightarrow$ Output(Softmax)

- 손 글씨에 해당하는 이미지에 필터를 적용
- 합성곱, 풀링 과정을 반복하여 패턴 학습
- 최종적으로 FC 레이어를 통해 추출된 특징을 이용해 클래스 점수를 계산해 확률로 변환

AI Modeling 기본

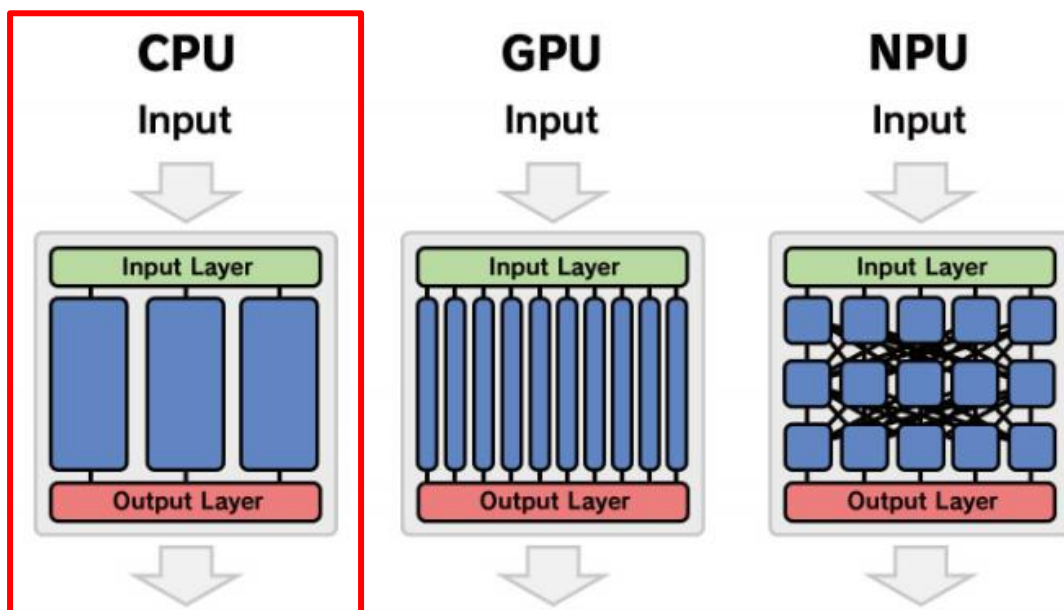


8. LeNet-5의 구조

- Lenet 5가 학습한 필터
- 해당 필터들을 통해 입력 이미지가 들어오면 특징을 추출
- 추출된 특징들을 바탕으로 어떤 숫자인지 분류

tc-nn-toolkit 이해

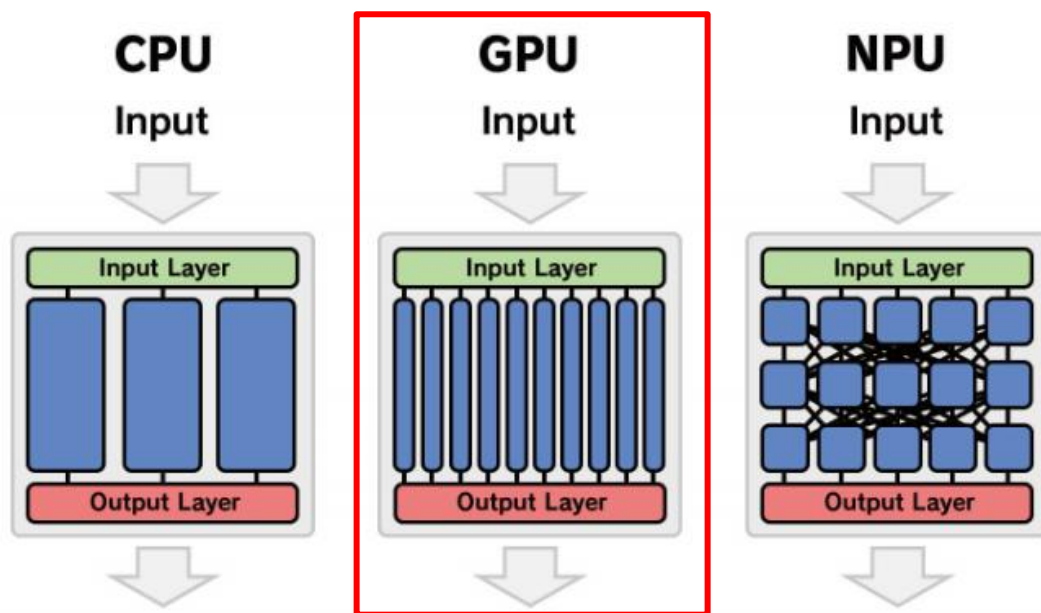
CPU vs NPU vs GPU



- 순차적 처리에 최적화된 장치로 한 번에 적은 수의 작업을 빠르게 처리
- 모델 학습 및 추론에서 병렬 처리에는 제한이 있어 대규모 연산에는 비효율적

tc-nn-toolkit 이해

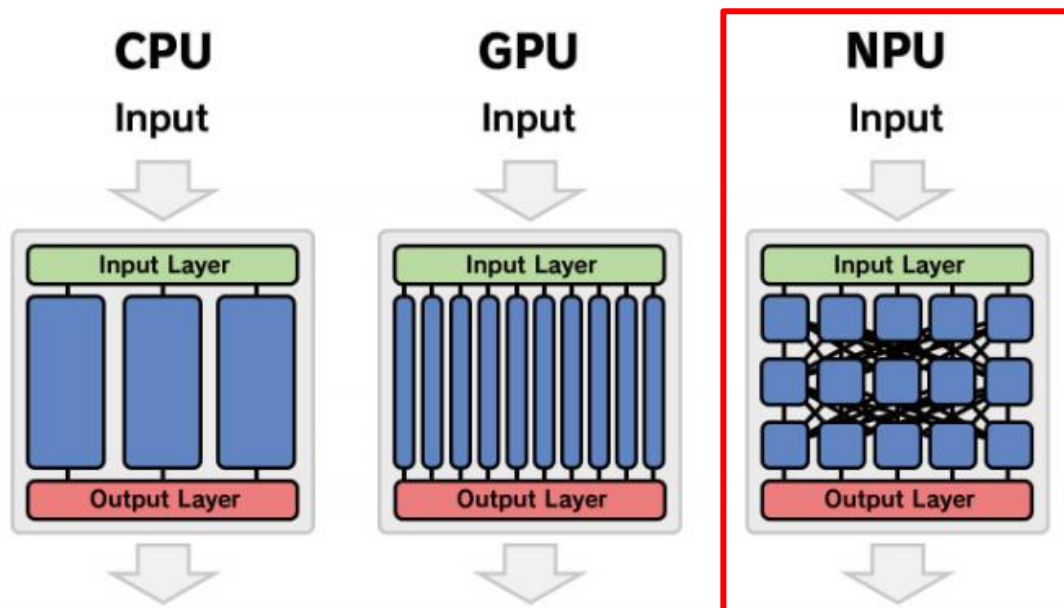
CPU vs NPU vs GPU



- 대규모 병렬 연산에 최적화된 구조, 수천 개의 작은 연산을 동시에 처리
- 복잡한 수학적 연산을 빠르게 처리할 수 있어 AI 모델의 학습과 추론에 널리 사용

tc-nn-toolkit 이해

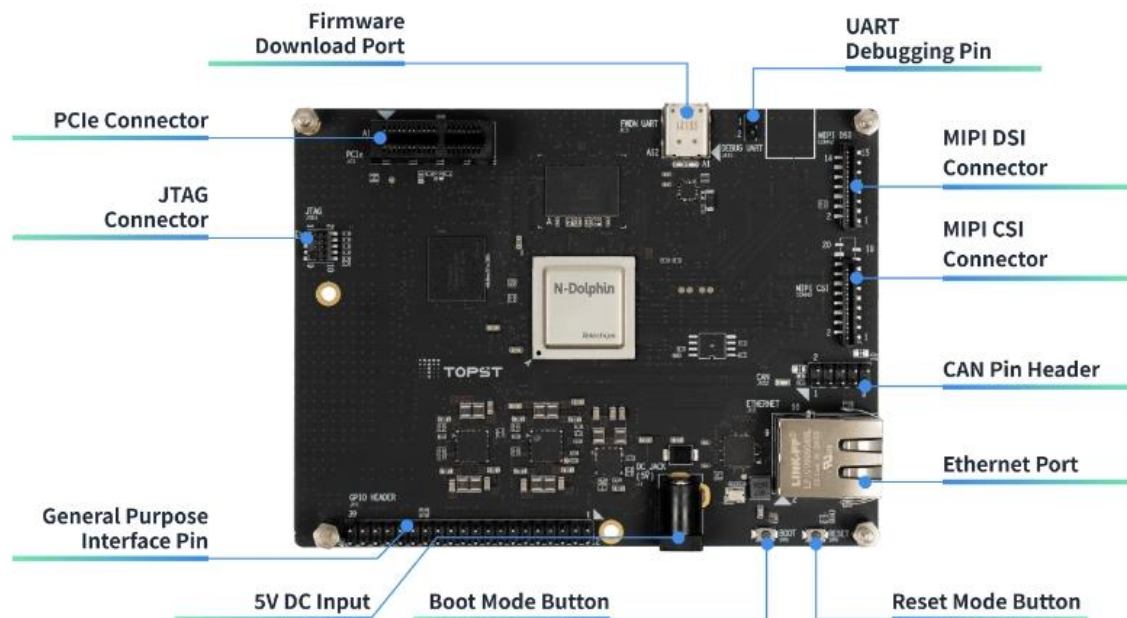
CPU vs NPU vs GPU



- AI 연산에 특화된 하드웨어로, 매트릭스 연산과 텐서 처리에 최적화
- 고속 연산, 저전력을 요구하는 임베디드 시스템에서 주로 사용
- CNN, RNN 과 같은 딥러닝 모델 처리에 매우 효율적

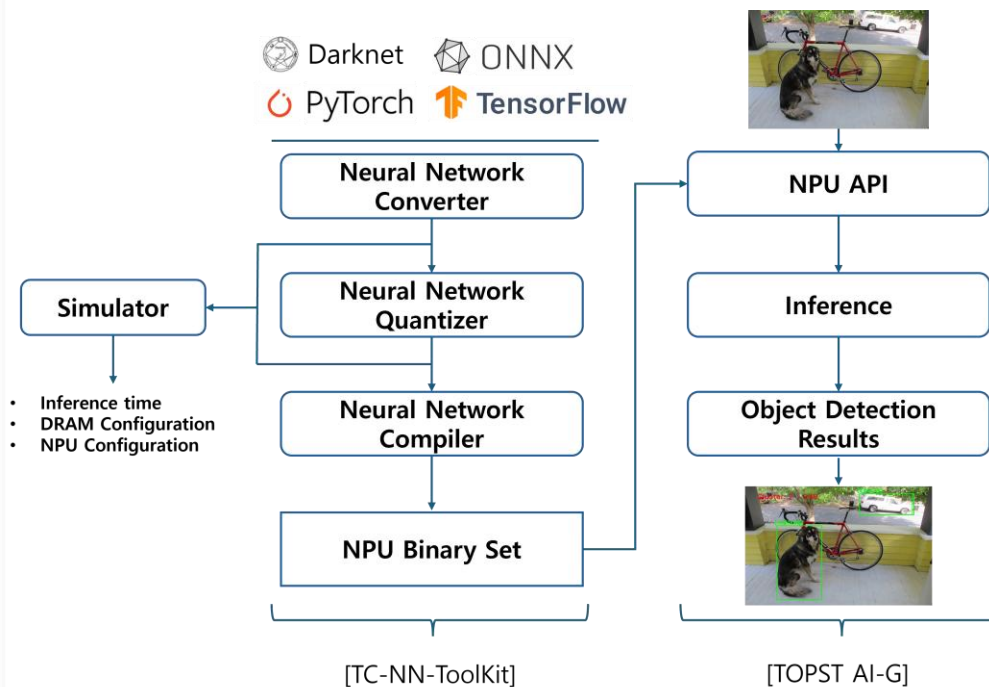
tc-nn-toolkit 이해

TOPST AI-G NPU



- 4TOPS + 4TOPS 듀얼 클러스터
- CNN과 같은 이미지 처리 특화
- 저전력, 고성능 NPU
- 동시에 2가지 모델 추론
- CPU가 내장되어 있어 stand alone으로 모델 추론 및 시각화

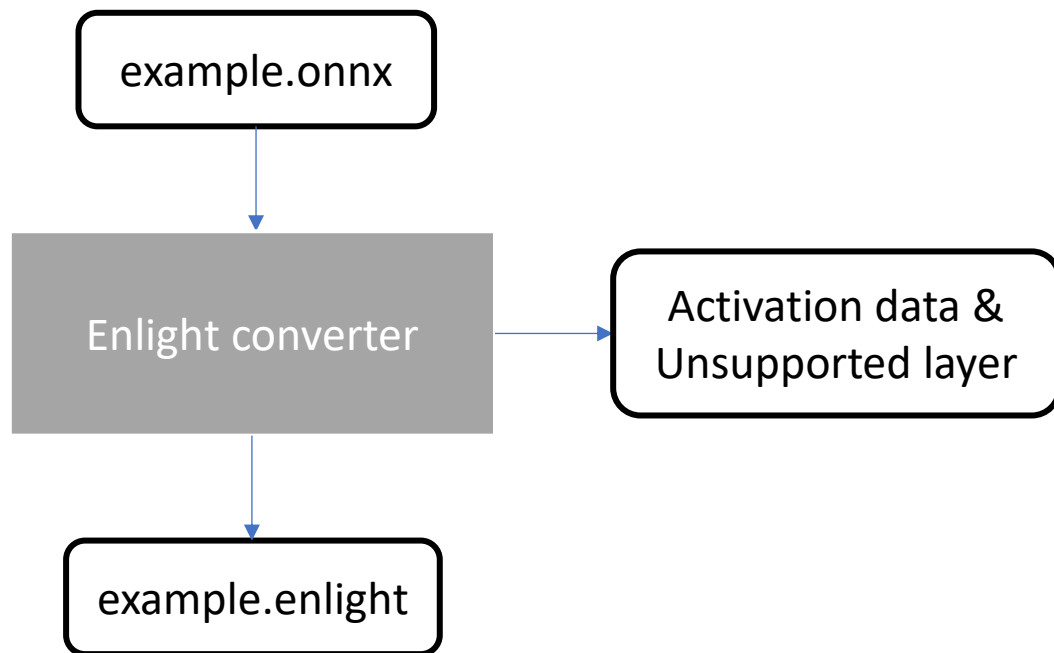
tc-nn-toolkit 이해



1. tc-nn-toolkit 소개

- NPU에서 구동하기 위한 모델 변환 툴
- 4개의 AI 프레임 워크 모델 변환 지원
- Convert, quantize, compile 총 3단계로 구성
- Simulator를 통해 양자화 전-후 모델 평가
- float32 모델을 int8 모델로 최적화

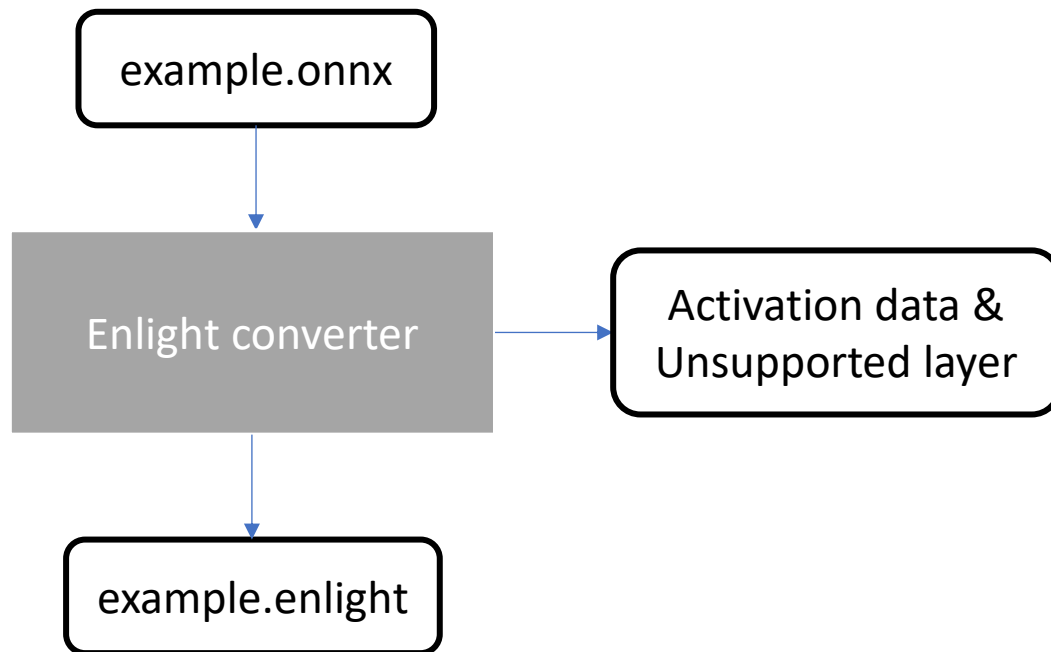
tc-nn-toolkit 이해



1. converter 이해

- Pytorch, TensorFlow, Darknet, ONNX와 같은 프레임 워크에서 학습된 모델을 변환
- TOPST AI-G NPU에서 추론을 위한 최적화된 포맷으로 변환
- 변환하는 모델의 지원되지 않는 연산자 확인
- 양자화를 위한 활성화 데이터 통계 수집

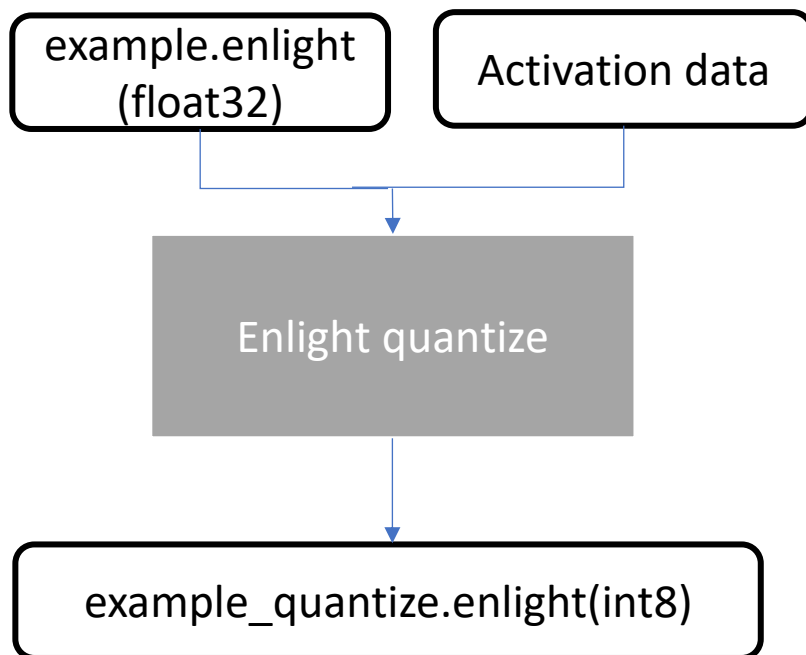
tc-nn-toolkit 이해



1. converter 이해

- 명령어 사용 방식
- Python `convert.py --{사용 옵션}`
- 주요 옵션
 - type - 후처리 타입
 - enable-track - 활성화 통계 수집 여부
 - Enable-letterbox - letterbox 활성화 여부
 - dataset - 활성화 통계 수집에 사용할 데이터셋
 - num-class - 클래스 수
- 이외에도 다수 옵션 존재

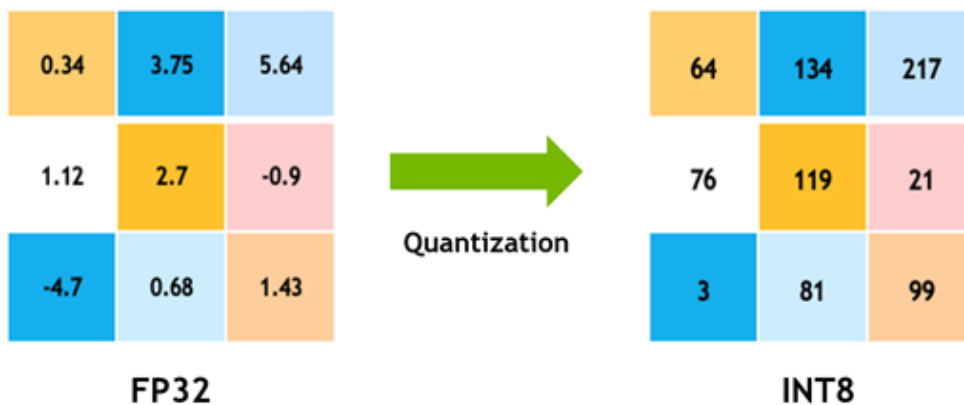
tc-nn-toolkit 이해



2. quantize 이해

- 부동소수점 연산을 정수 연산으로 변환
- Float32 모델을 int8 모델로 변환하여 모델의 크기를 줄이고 속도 향상
- 가중치와 활성화를 8bit 정수로 변환
- 활성화 데이터 통계를 통해 최적의 양자화 비율을 계산하여 적용

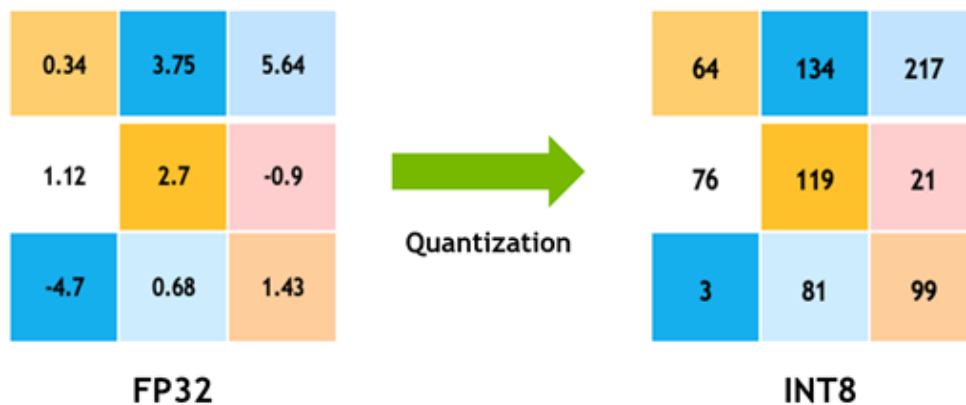
tc-nn-toolkit 이해



2. quantize 이해

- FP32 : 32비트 부동 소수점 형식.
 - 높은 정밀도를 제공
 - 메모리 공간을 다수 차지
 - 연산이 느림
- INT8 : 8비트 정수점 형식
 - 저장 공간과 연산속도에서 유리
 - 정밀도가 낮아질 수 있음
 - 저전력을 요구하는 환경에서 효율적 처리

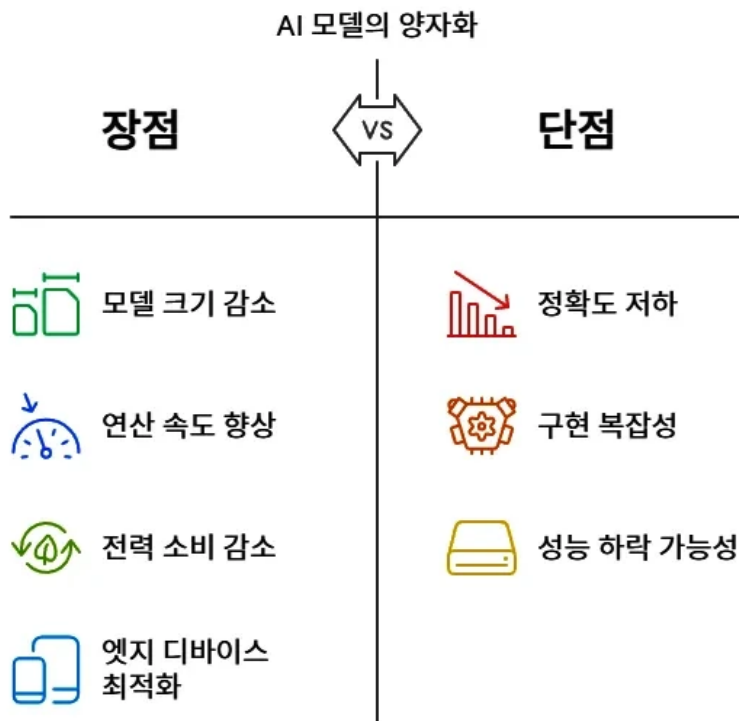
tc-nn-toolkit 이해



2. quantize 이해

- 양자화란?
 - 정수화를 통해 모델의 가중치, 활성화 값을 정수로 변환하는 작업
- 가중치 양자화
 - 모델의 가중치 매개변수를 정수 변환
 - 양자화 스케일을 사용해 정수 변환
- 활성화 양자화
 - 신경망의 각 층의 출력 값을 정수 변환
 - 입력 데이터 범위와 스케일을 고려해 양자화된 값으로 변환

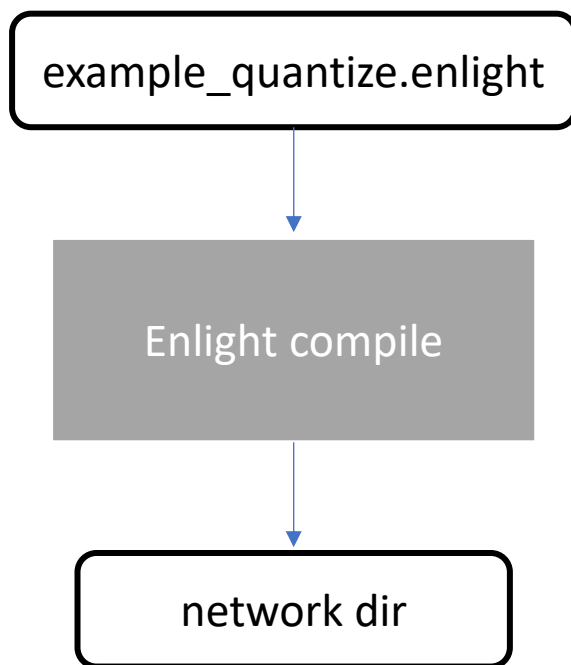
tc-nn-toolkit 이해



2. quantize 이해

- 장점
 - 모델 크기 감소 : 모델의 크기가 약 4배 줄어듦
 - 연산 속도 향상
 - 메모리 사용 절감
- 단점
 - 정밀도를 낮추기 때문에 정확도 손실

tc-nn-toolkit 이해

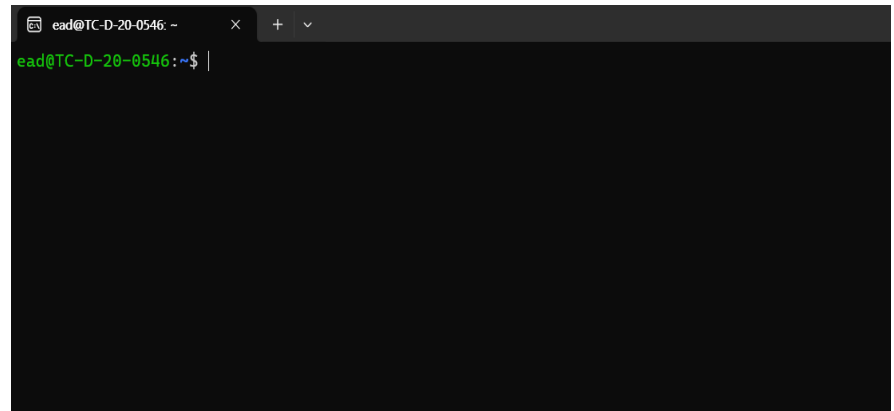
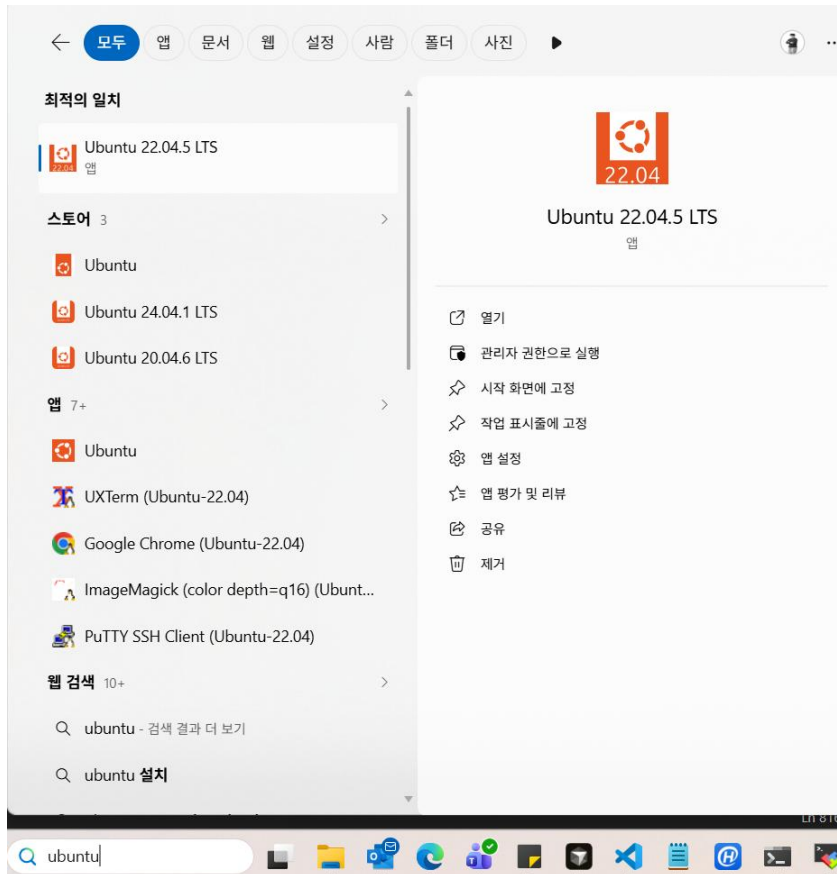


3. compile 이해

- 양자화된 신경망 모델을 NPU 명령어와 네트워크 파라미터로 변환하여 NPU에서 실행 가능한 형식으로 컴파일
- NPU 명령어 파일과 양자화된 가중치 등 생성
- 후처리 함수를 위한 소스 생성
- 이후 CPU에서 추론된 텐서에 대한 후처리 연산 가능

tc-nn-toolkit 이해

- WSL Ubuntu 22.04 실행



tc-nn-toolkit 이해

tc-nn-toolkit : [download link](#)

- tc-nn-toolkit 설치 (wsl ubuntu)
 - `$ cd ~/Work`
 - `$ wget -O tc-nn-toolkit.zip "https://topst-downloads.s3.ap-northeast-2.amazonaws.com/Education/Motrex/tc-nn-toolkit.zip"`
 - `$ unzip tc-nn-toolkit.zip`
 - `$ cd tc-nn-toolkit`
 - `$ chmod 755 *`

tc-nn-toolkit 환경 세팅

- 가상환경 세팅 & 필요 패키지 설치

- `$ sudo add-apt-repository ppa:deadsnakes/ppa -y`
- `$ sudo apt install python3.8 python3.8-venv python3.8-dev python3.8-tk`
- `$ ./setup_venv.sh`

- 가상환경 활성화

- `$ source venv/bin/activate`
- `$ deactivate`

tc-nn-toolkit 환경 세팅

- 호환되지 않는 GPU dri로 인해 에러가 발생할 경우(모델 학습, 변환 시)
 - 기존 가상환경 삭제 후 재 설치
 - `cd ~/Work/tc-nn-toolkit`
 - `python3.8 -m venv ./venv`
 - `source ./venv/bin/activate`
 - `pip install --upgrade pip`
 - `pip install -r requirements.txt`
 - `pip install torch==1.12.0+cpu torchvision==0.13.0+cpu torchaudio==0.12.0 --extra-index-url \https://download.pytorch.org/whl/cpu`
 - `pip install ./enlight_viewer/netron-3.5.4-py2.py3-none-any.whl`

tc-nn-toolkit 환경 세팅

- Toolchain (gcc-arm-9.2) install
 - `$ cd`
 - `$ pwd`
`/home/user`
 - `$ wget https://developer.arm.com/-/media/Files/downloads/gnu-a/9.2-2019.12/binrel/gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz`
 - `$ tar -xvf gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz`
 - `$ echo "export PATH=~/.gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu/bin:\$PATH" >> ~/.bashrc`
 - `$ source ~/.bashrc`
 - `$ aarch64-none-linux-gnu-gcc --version`

Lenet 모델 학습

- Lenet5 모델 학습 코드 작성
 - Lenet 모델 학습용 디렉터리 생성
 - `cd ~/Work/tc-nn-toolkit`
 - `$ mkdir lenet_dir && cd lenet_dir`
 - vi 편집기로 lenet 학습 코드 작성
 - `$ vi train_lenet.py`

Lenet 모델 학습

- Lenet5 모델 학습 코드 작성

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# LeNet 모델 정의 (기존과 동일)
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, kernel_size=5, padding=2) # 컬러 이미지를 위한 조정
        (3 채널)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, padding=2)
        self.conv3 = nn.Conv2d(16, 32, kernel_size=3, padding=2) # Conv3 레이어 추가 (커
        널 크기 변경)
        self.fc1 = nn.Linear(32 * 5 * 5, 120) # 32 * 1 * 1으로 입력 차원 조정
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10) # 11개의 클래스 출력 (10개의 숫자 + 1개의 별)
```

Lenet 모델 학습

- Lenet5 모델 학습 코드 작성

```
def forward(self, x):
    x = torch.relu(self.conv1(x))
    x = torch.max_pool2d(x, 2)
    x = torch.relu(self.conv2(x))
    x = torch.max_pool2d(x, 2)
    x = torch.relu(self.conv3(x))
    x = torch.max_pool2d(x, 2)
    x = x.view(-1, 32 * 5 * 5)
    x = torch.relu(self.fc1(x))
    x = torch.relu(self.fc2(x))
    x = self.fc3(x)
    return x

# 데이터 전처리 정의 (MNIST를 3채널로 맞춤)
transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=3), # 그레이스케일을 3채널로 변환
    transforms.Resize((32, 32)), # 32x32 크기로 조정
    transforms.ToTensor(),
    transforms.Normalize((0.1307, 0.1307, 0.1307), (0.3081, 0.3081, 0.3081))
])

# MNIST 데이터셋만 로드
train_dataset = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True,
transform=transform)
```

Lenet 모델 학습

- Lenet5 모델 학습 코드 작성

```
# 데이터로더
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)

# 모델, 손실 함수, 최적화기 설정
device = torch.device("cpu")
model = LeNet().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# 훈련 함수
def train(model, device, train_loader, optimizer, criterion, epoch):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % 100 == 0:
            print(f"훈련 Epoch: {epoch} [{batch_idx * len(data)} / {len(train_loader.dataset)}] ({100. * batch_idx / len(train_loader):.0f}%) \t 손실: {loss.item():.6f}")
```

Lenet 모델 학습

- Lenet5 모델 학습 코드 작성

```
# 평가 함수
def test(model, device, test_loader, criterion):
    model.eval()
    test_loss = 0
    correct = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            test_loss += criterion(output, target).item()
            pred = output.argmax(dim=1, keepdim=True)
            correct += pred.eq(target.view_as(pred)).sum().item()

    test_loss /= len(test_loader.dataset)
    accuracy = 100. * correct / len(test_loader.dataset)
    print(f"\n테스트 세트: 평균 손실: {test_loss:.4f}, 정확도: {correct}/{len(test_loader.dataset)} ({accuracy:.0f}%)")
```

Lenet 모델 학습

- Lenet5 모델 학습 코드 작성

```
# 훈련 및 평가
num_epochs = 10
for epoch in range(1, num_epochs + 1):
    train(model, device, train_loader, optimizer, criterion, epoch)
    test(model, device, test_loader, criterion)

# 모델을 ONNX 형식으로 내보내기
dummy_input = torch.randn(1, 3, 32, 32, device=device)
onnx_filename = "lenet_with_conv3.onnx"
torch.onnx.export(model, dummy_input, onnx_filename, input_names=['input'], output_names=['output'])
print(f"모델이 {onnx_filename} 파일로 내보내졌습니다.")
```

Lenet 모델 학습

- Lenet5 모델 학습
 - `$ cd ~/Work/tc-nn-toolkit`
 - `$ . venv/bin/activate`
 - `$ cd lenet_dir`
 - `$ python3 train_lenet.py`

Lenet 모델 변환

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit$ python ./EnlightSDK/converter.py \
./input/lenet_with_conv3.onnx \
--type class \
--dataset Custom \
--dataset-root my_dataset_path \
--output ./output_networks/lenet_mnist.enlight \
--enable-track \
--num-class 10

[INFO] Find model_config lenet_with_conv3.json, but can't find it
[INFO] so, set DEFAULT config parameter

*****
[v0.9.9] Converting to enlight format. start.
*****
model          : ./input/lenet_with_conv3.onnx
output         : ./output_networks/lenet_mnist.enlight
model_config   : auto
type          : class
```

SUMMARY	
Number of compatible layers	11
Number of incompatible layers	0
Total number of layers	11

```
Checking compatibility with ENLIGHT NPU ... Done
Writing compatibility result to ... /home/ead/tc-nn-toolkit/t
y_results/lenet_with_conv3_compatibility.log

Serializing Graph done.
Writing to File..... Done
Writing File path : ./output_networks/lenet_mnist.enlight

Converter. done.
```

Lenet 모델 컨버팅

```
$ cd ..
```

```
$ python ./EnlightSDK/converter.py \
./lenet_dir/lenet_with_conv3.onnx \
--type class \
--dataset Custom \
--dataset-root my_dataset_path \
--output ./output_networks/lenet.enlight \
--enable-track \
--num-class 10
```

Lenet 모델 변환

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit$ python ./EnlightSDK/quantizer.py
./output_networks/lenet_mnist.enlight --output ./output_networks/lenet_mnist_quantized.e
nlight

[INFO] Find model_config lenet_mnist.json, but can't find it
[INFO] so, set DEFAULT config parameter

*****
Quantization. start
*****
model                : ./output_networks/lenet_mnist.enlight
output               : ./output_networks/lenet_mnist_quantized.enlight
model_config         : auto
stats_file           : None
scales_file          : None
qbits_file           : None

```

```
Start BatchnormalizeFold optimizer
optimizing ..... done.
Start FuseConstantEltwLayer optimizer
optimizing ..... done.
Start TreatIntermediateOuput optimizer
optimizing ..... done.
Start ModifyLayerQuantizationFriendly optimizer
optimizing ..... done.
quantization..... done.
End sanity check for custom quantization
Start MakeCompilerFriendly optimizer
optimizing ..... done.
Start MakeLayerAligned optimizer
optimizing ..... done.
Serializing Graph done.
Writing to File..... Done
Writing File path : ./output_networks/lenet_mnist_quantized.enlight
Quantizer done.
```

Lenet 모델 양자화

```
$ python ./EnlightSDK/quantizer.py \
./output_networks/lenet.enlight \
--output ./output_networks/lenet_quantized.enlight
```

Lenet 모델 변환

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit$ python ./EnlightSDK/compiler.py \
/output_networks/lenet_mnist_quantized.enlight --th-iou 0.5 --th-conf 0.5

[INFO] Find model_config lenet_mnist.json, but can't find it
[INFO] so, set DEFAULT config parameter

*****
Compiling network. start.
*****
model                : ./output_networks/lenet_mnist_quantized.enlight
dump_root            : ./output_code
model_config         : auto
th_conf              : 0.5
th_iou               : 0.5
max_detection        : -1
batch_mode           : False
```

```
code-emitting ..... done.
Start LinkIact(version=NPUV20) code-emitting
code-emitting ..... done.
work-buffer allocation sanity check ..... done.
Start MakeCode(version=NPUV20) code-emitting
code-emitting ..... done.

./output_code/lenet_mnist_quantized/npv_cmd.h created
./output_code/lenet_mnist_quantized/npv_cmd.bin created
./output_code/lenet_mnist_quantized/npv_cmd.txt created

Writing [43776]/[43776]
./output_code/lenet_mnist_quantized/quantized_network.bin created
./output_code/lenet_mnist_quantized/oact_entry_table.py created
./output_code/lenet_mnist_quantized/network.h created
./output_code/lenet_mnist_quantized/post_process.c created
./output_code/lenet_mnist_quantized/oe_network.bin created
./output_code/lenet_mnist_quantized/oe_network.h created

NPU compiler: done sucessfully.

Compiler. done.
```

Lenet 모델 컴파일

```
$ python ./EnlightSDK/compiler.py \
./output_networks/lenet_quantized.enlight \
--th-iou 0.5 --th-conf 0.5
```

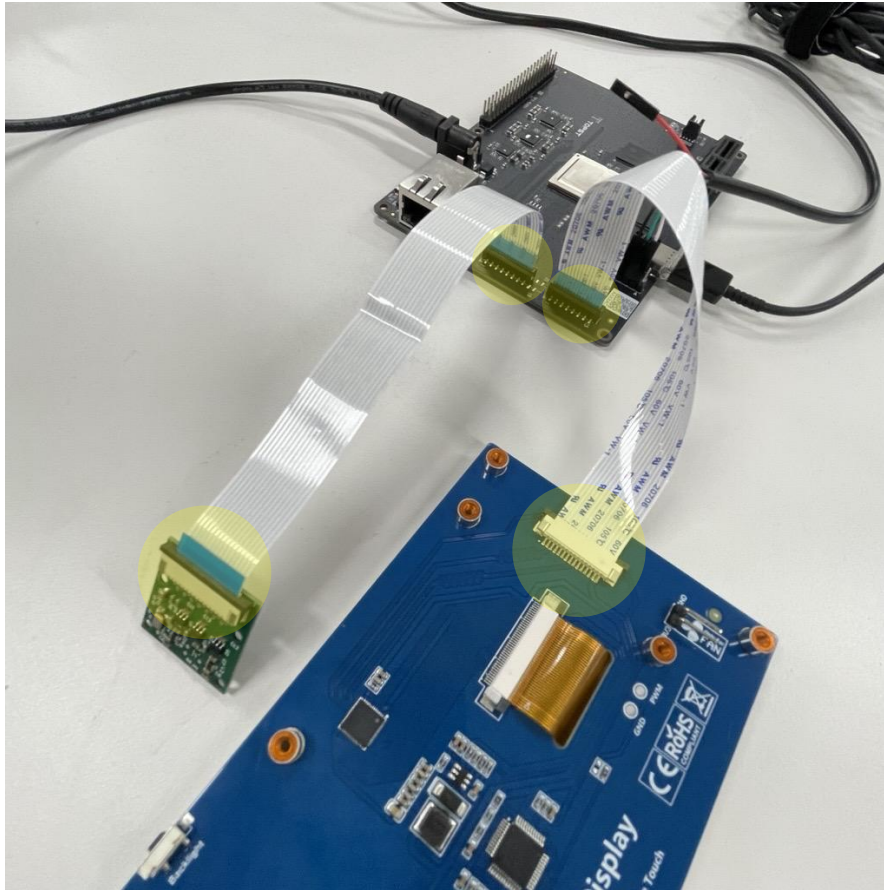
Lenet 모델 변환

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit/build_network$ make
aarch64-none-linux-gnu-gcc -fPIC -Wall -D__NO_INLINE__ -std=gnu99 -lpthread -pthread -
02 -g -D__i386__ -pg -Wno-unused -Wno-attributes -fno-delete-null-pointer-checks -fP
IC -no-pie -Ienlight -Inetwork -c post_process.c classifier/classifier.c detector/detec
tor.c detector/yolo_detector.c detector/ssd_detector.c custom_postproc/custom_postproc.
c enlight/enlight_network.c network.c
detector/yolo_detector.c: In function 'detect_object_float':
detector/yolo_detector.c:810:15: warning: 'obj.dims[2]' may be used uninitialized in thi
s function [-Wmaybe-uninitialized]
   810 |         const int len_obj = obj[0].dims[2];
       |         ^~~~~~
aarch64-none-linux-gnu-gcc -shared -Wl,-soname,net.so -o net.so post_process.o detector.
o classifier.o yolo_detector.o ssd_detector.o custom_postproc.o enlight_network.o netwo
rk.o
rm -rf ./*.o network/*.o enlight/*.o /*.o
```

Lenet 모델 빌드

```
$ cd build_network
$ cp -r ../output_code/lenet_quantized/ ./ -ar
$ rm -rf network.h post_process.c
$ ln -s lenet_quantized/network.h
$ ln -s lenet_quantized/post_process.c
$ make
$ cp net.so lenet_quantized/
$ scp -r lenet_quantized root@192.168.0.100:/home/root/
-> yes -> root 입력
```

모델 추론 – Lenet5



MIPI 장치간 연결 주의사항

1. 반드시 화면과 같이 연결
 - FFC 케이블 부착 방향 주의
2. 디바이스 연결 후에 AI-G 전원 연결

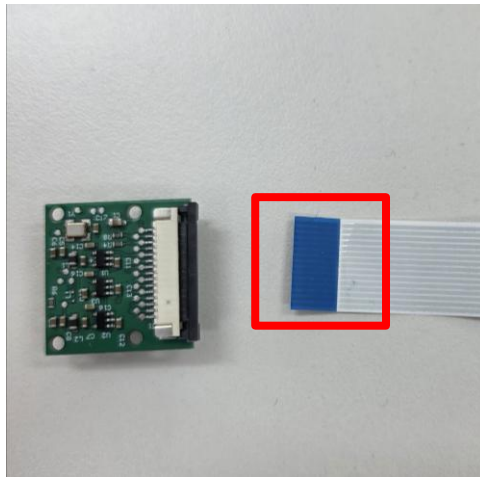
모델 추론 - Lenet5

1. 고정 핀 해제

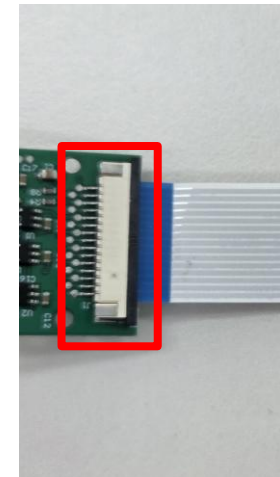
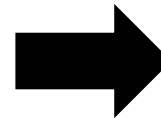
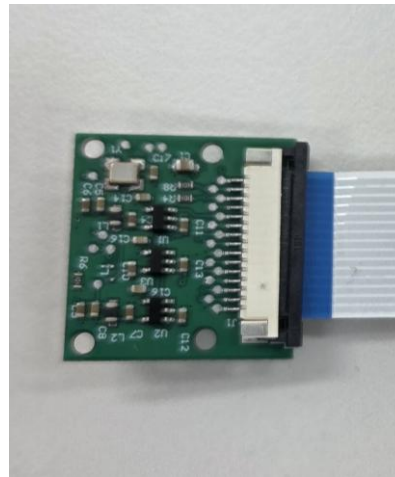
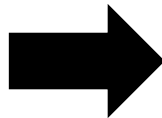


모델 추론 - Lenet5

2. 케이블 연결



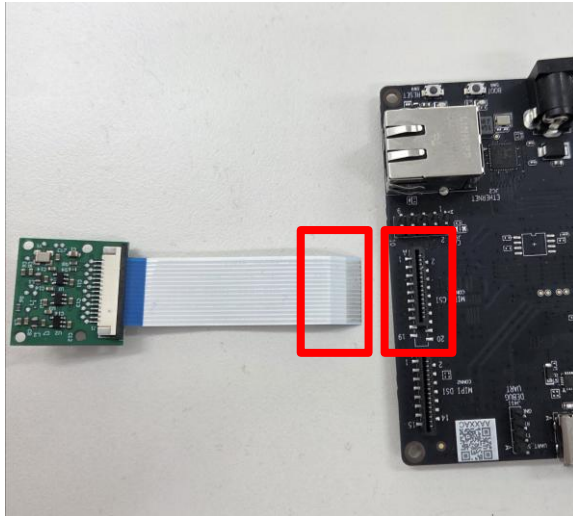
케이블 방향 확인!



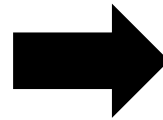
핀 고정

모델 추론 – Lenet5

3. 카메라와 보드 연결



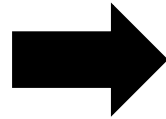
케이블 방향 확인!



카메라와 보드 연결

모델 추론 - Lenet5

1. 고정 핀 해제

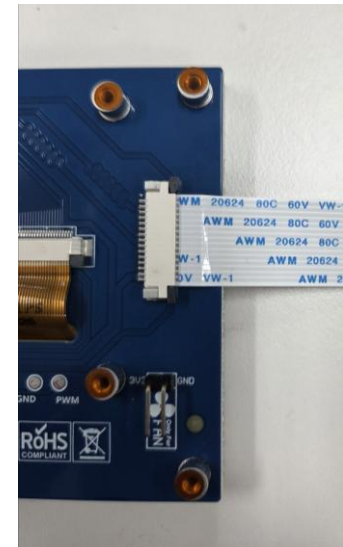
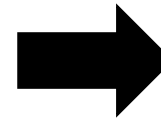
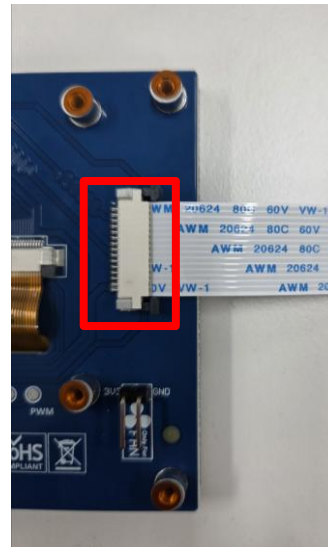
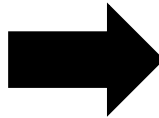


모델 추론 - Lenet5

2. 케이블 연결



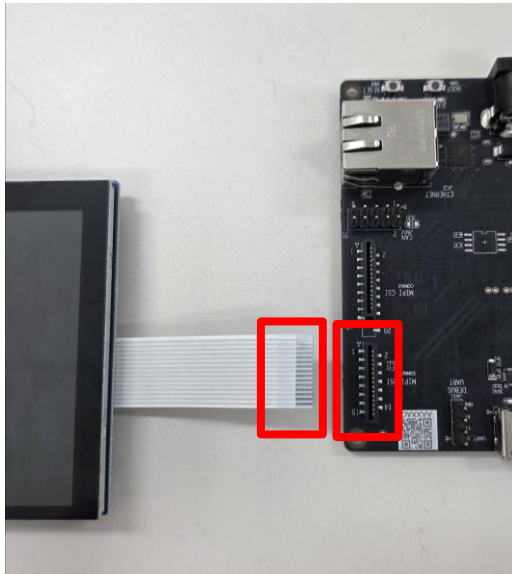
케이블 방향 확인!



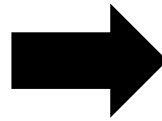
핀 고정

모델 추론 – Lenet5

3. 보드 연결

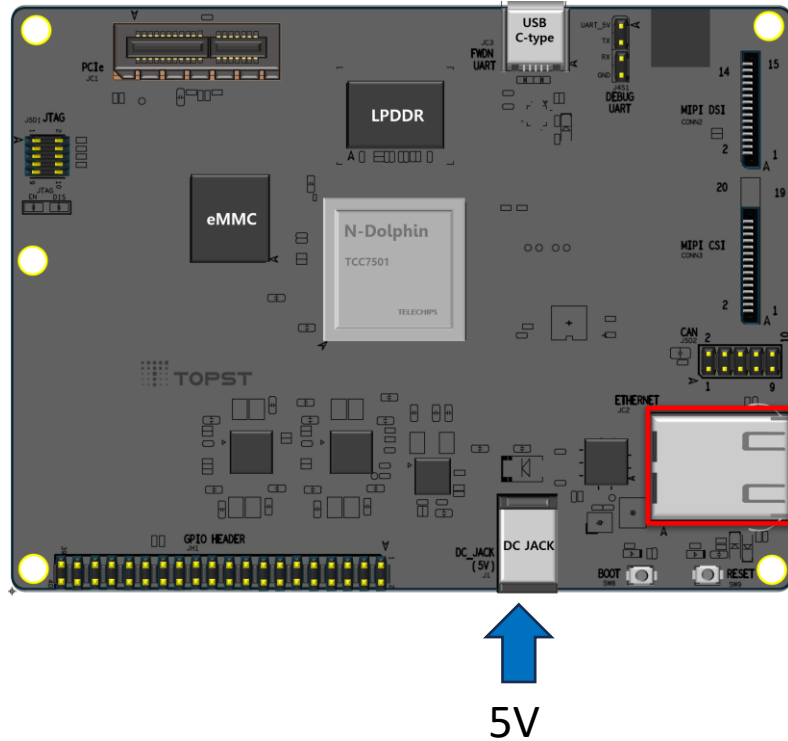


케이블 방향 및 MIPI 포트 위치 확인!



보드와 연결

모델 추론 – Lenet5

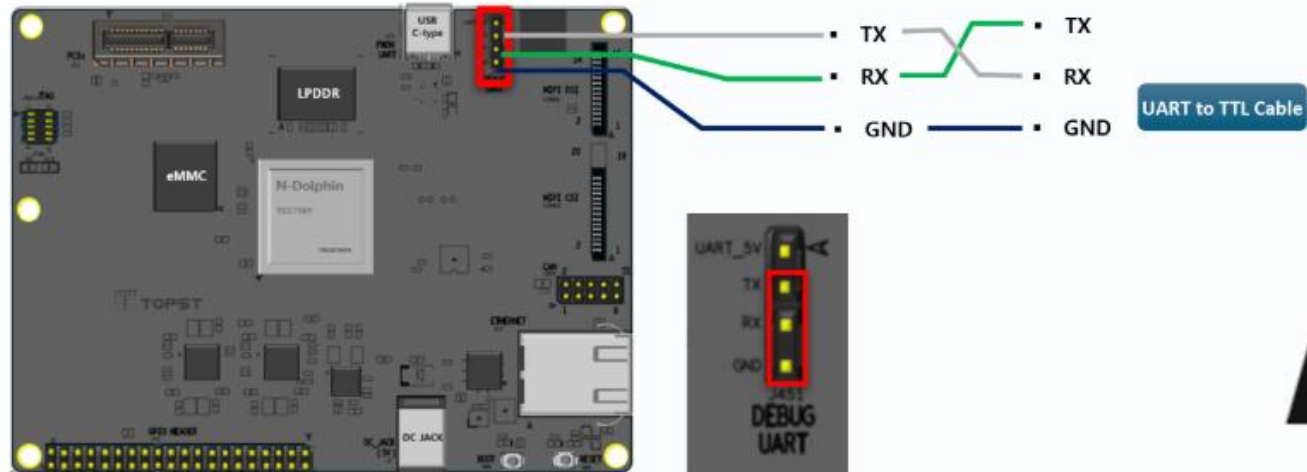


Ethernet Cable



- 이더넷 및 5V 전원 연결

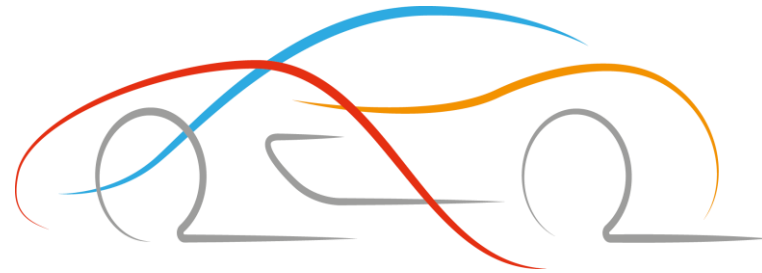
모델 추론 - Lenet5



- UART 디버거 연결

```
$ ls
lenet_quantized
$ tcnnapp -n lenet_quantized -p /dev/video2
```

추론 결과 확인 - 종이에 손 글씨 작성 (0~9)



Thank You